Politechnika Częstochowska

Wydział Informatyki i Sztucznej Inteligencji

PRACA DYPLOMOWA INŻYNIERSKA

System dekoracji wnętrz z wykorzystaniem technologii wirtualnej rzeczywistości (VR)

Interior decoration system using virtual reality technology (VR)

Andrzej Paruch

Nr albumu: 136606

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: I

Promotor pracy: dr hab. Inż. Krystian Łapa

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, ……

Spis treści

1. Wstęp 5
1.1 Wirtualne Projektowanie Wnętrz 5
1.2 Wirtualna Rzeczywistość 8
1.3 Zastosowanie 23
2. Część Teoretyczna 24
2.1 Dziedzina Pracy 24
2.2 Dobór Technologii VR 24
2.3 Unreal Engine 5 30
2.4 Założenia i Funkcjonalność aplikacji 36
3. Część Praktyczna 40
3.1 Implementacja Wirtualnej Rzeczywistości 40
3.2 Generowanie Pomieszczenia 54
3.3 Tablet 65
3.4 Tworzenie Przedmiotów 65
3.5 Zmiana Wyglądu Ścian 65
3.6 Stałe Obiekty 65
3.7 Ustawienia 65

1. Wstęp

Jedną z podstawowych rzeczy jaką trzeba wykonać podczas planowania dekoracji wnętrza jest jej wizualizacja. Stworzenie wirtualnego środowiska, gdzie można zobaczyć jak będzie wyglądać przyszły dom lub pokój. W dobie internetu oraz komputerów człowiek ma dostęp do różnego rodzaju oprogramowania, które ma mu służyć jako pomoc w projektowaniu wnętrz. Jednak ~~zawsze~~ ta wizualizacja jest mało przejrzysta, bo to co widzimy to obrazek na ekranie komputera. Wygodniej by było wejść do tego wirtualnego świata i na własne oczy zobaczyć jak zaprojektowane pomieszczenie wygląda. Tu z pomocą przychodzi technologia wirtualnej rzeczywistości umożliwiająca wstąpić w cyfrowy świat.

1.1 Wirtualne Projektowanie Wnętrz

Ważnym punktem podczas remontu, czy planowaniu nowego domu jest zwizualizowanie jak to wszystko będzie wyglądać. Pomaga to w zaplanowaniu zakupów potrzebnych rzeczy, ale i również w pokazaniu jak całość się będzie prezentować. Wygodnym sposobem na zaprojektowanie wnętrza jest użycie specjalistycznych programów, które są wstanie pokazać użytkownikowi jak jego pokój będzie się prezentować. Ważnymi elementami takich programów jest łatwe i intuicyjne posługiwanie się nimi oraz oddanie skali pomieszczenia. Na rynku jest multum systemów służących do tego zadania.

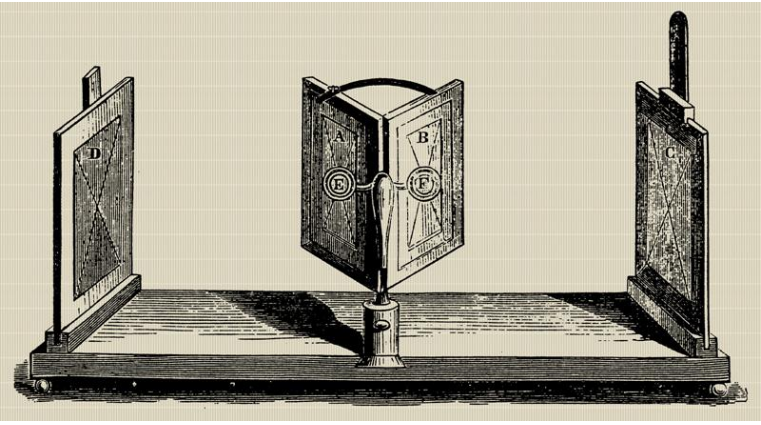
Tabela 1.1 Spis przykładowych programów do projektowania wnętrz

Nazwa	Forma płatności	Cena	Forma Wizualizacji	Oferowane Usługi
Homestyler	Abonament	Darmowy/$19.6	Flat/VR	Duża baza mebli, Konwertowanie planow 2D na 3D, Prezentacja VR, Moduł AI
Sweet Home 3D	-	Darmowy	Flat	Spora biblioteka mebli, możliwość tworzenia własnych wtyczek
Sketch Up	Zakup licencji na rok	1 786,73 zł-5 136 zł	Flat	Baza gotowych modeli, Tworzenie modeli w oparciu o zdjęcia, Baza wtyczek do programu
Live Home 3D	Zakup Jednorazowy	Darmowy/$99.99	Flat	Baza gotowych modeli, importowanie własnych modeli, Dostępne na urządzenia mobilne
p.Con planner	Zakup Jednorazowy	Darmowy/19 999 zl	Flat/VR	Baza gotowych modeli, Wtyczka VR

Rozwiązania proponowane przez inne firmy często opierają się na opracowaniu planów 2D i późniejszym rozstawieniu mebli w przestrzeni trójwymiarowej. Oferują tworzenie renderów w wysokiej rozdzielczości, ale i również spacery po zaprojektowanych wnętrzach. Niektóre systemy nawet mają wtyczki pozwalające na zobaczenie pomieszczeń w wirtualnej rzeczywistości. Jednak sposób faktycznego projektowania jest praktycznie identyczny dla każdego z tych programów. To co je wyróżnia to oferowane usługi oraz przejrzystość interfejsu użytkownika. Jednakże nawet najprostsze oprogramowanie wymaga wstępnego zapoznania się ze środowiskiem. Nie jest to coś, co zachęca użytkowników, którzy chcą jednorazowo zaplanować swoje mieszkanie. Takie programy są dla ludzi, którzy zajmują się tą tematyką zawodowo lub hobbystycznie. Dla użytkownika, który nie jest zaznajomiony z tą tematyką i chcę tylko zaprojektować swój pokój, zwizualizować swój pomysł to jest bardzo nie przystępne. Choć te programy oferują bogatą funkcjonalność to brakuje im intuicyjności i przystępności. Użytkownik powinien móc szybko i intuicyjnie zaprojektować wnętrze bez wczytywania się w długie strony dokumentacji. Jednakże brak przystępności wynika głównie ze specyfiki urządzeń na jakie są produkowane te systemy. Komputery osobiste pozwalają na obsługiwanie dość skomplikowanych programów, co jest ich sporą zaletą, ale dla nowego użytkownika jest to przytłaczające. Trzeba nauczyć się posługiwać interfejsem co potrafi być żmudnym zajęciem. To jest cena dużej funkcjonalności, która dla osoby zajmującej się tą tematyką jest warta zapłacenia. Problem ma użytkownik, który albo zaczyna swoją przygodę z projektowaniem wnętrz lub potrzebuje szybko zwizualizować pomysł i nie ma czasu na naukę całego programu. Brakuje systemu, który dzięki swojej prostocie oraz intuicyjności jest w stanie oferować szybką wizualizację projektu oraz pomóc w zaprojektowaniu wnętrza. Taki program powinien być użyteczny dla zwykłego użytkownika, który nie ma doświadczenia z dekorowaniem wnętrz, a chcę zaprojektować swoje mieszkanie. By było to możliwe to sam proces projektowania musi być naturalny dla użytkownika i responsywny. Użytkownik musi w czasie rzeczywistym widzieć swoje działania. Niestety taki system musiałby zrezygnować z dużej precyzji oraz specyficznej funkcjonalności, kosztem wygody dla użytkownika. Wirtualne projektowanie wnętrz jest rzeczą bardzo potrzebną, ale aktualne rozwiązania wymagają dużej ilości czasu, by móc z nich korzystać w poprawny sposób.

1.2 Wirtualna Rzeczywistość

Wirtualna rzeczywistość, w skrócie VR, to symulacja doświadczenia prawdziwego świata przy pomocy technologii komputerowej. Ma ona symulować trójwymiarowy świat poprzez umieszczenie użytkownika wewnątrz niego. Jest to wykonywane poprzez odpowiednie gogle zakładane na głowę oraz różnego rodzaju kontrolery, które mają przechwytywać ruch użytkownika i przenosić je do wirtualnego świata. Zamiast być obserwatorem tego co się dzieje na ekranie, użytkownik jest w pełni czynnym uczestnikiem tego. Wirtualna rzeczywistość ma za zadanie dać użytkownikowi najbardziej naturalną i intuicyjną interakcje z nią. VR można łatwo pomylić z AR, MR, czy XR. AR to rozszerzona rzeczywistość, gdzie na realny świat są nakładane cyfrowe obiekty. W przeciwieństwie do VR to rozszerzona rzeczywistość całą swoją magię opiera na wzbogaceniu realnego świata, gdzie w przypadku wirtualnej rzeczywistości jest tworzony zupełnie osobny świat. Połączeniem tych dwóch technologii jest MR, czyli rzeczywistość mieszana. Tak jak w AR na realny świat są nakładane cyfrowe obiekty, ale można z nimi wejść w interakcje na poziomie VR. Można to określić jako bardziej interaktywny AR. XR jest terminem, który określa całe spektrum tych technologii. To co jest wspólnym czynnikiem XR jest danie użytkownikowi nowych doświadczeń rzeczywistości. Może się wydawać, że całe pojęcie XR jest dość nowe. Jednak pierwsze wzmianki o tym sięgają XIX wieku. Choć nie można było mówić o VR samym w sobie, to koncept oszukania mózgu, że to co widzi jest trójwymiarowe było z ludźmi od bardzo dawna. Jednym z pierwszych urządzeń, które miały imitować trójwymiar był stereoskop Wheatstone'a wynaleziony w 1838 roku.



Rys. 1.1 Stereoskop Wheatstone'a. Źródło: [4]

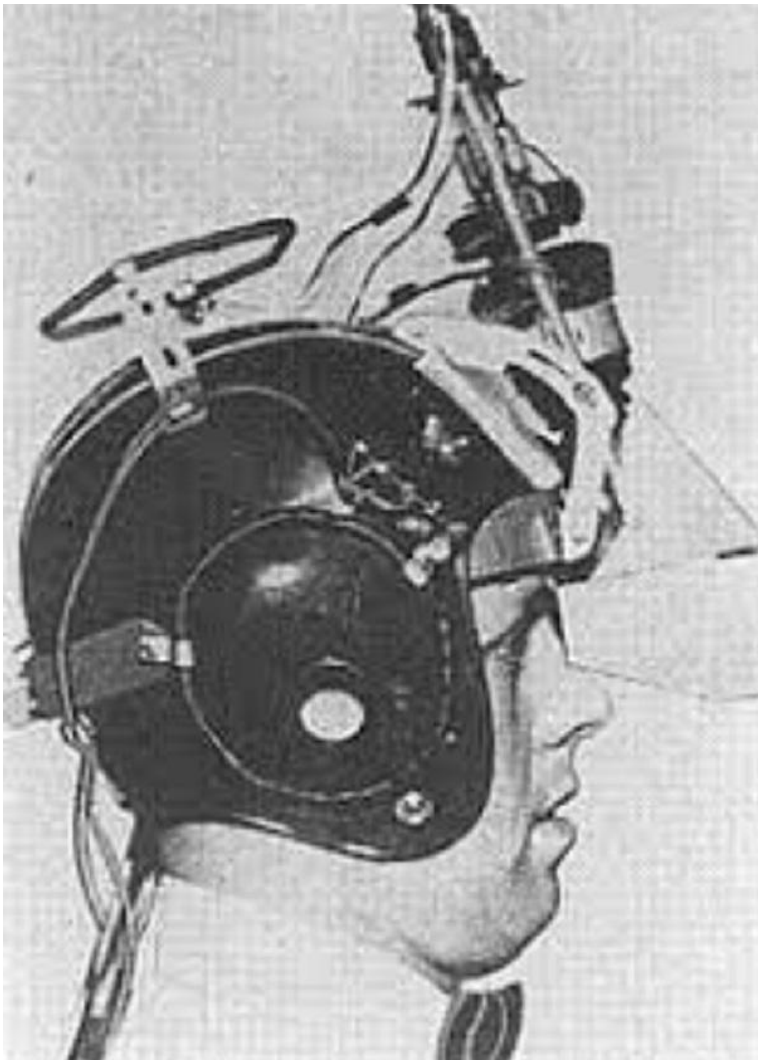
Urządzenie działało w prosty sposób. Naprzeciw sobie były ustawione dwa takie same obrazy. Pomiędzy nimi były lustra ustawione pod kątek 45 stopni względem obrazów. Obserwator mając założone szkła okularowe mógł obserwować trójwymiarowy obraz. To wszystko było iluzją polegająca na pokazaniu tego samego obrazu, ale pod innym kątem. Jedno oko widziało przedmiot inaczej niż drugie. Mózg to rejestrował i był oszukiwany, że widzi trójwymiarowy obiekt. Na tej zasadzie są oparte wszystkie urządzenia VR. Na iluzji obrazu 3D, poprzez wyświetlanie obrazu inaczej dla każdego oka. To jest tylko jeden zmysł, a co z pozostałymi? Technologia VR przecież polega na oszukiwaniu całego ciała, że znajduje się w wirtualnej przestrzeni. Na przestrzeni lat powstawało kilka urządzeń, które miały oszukiwać te zmysły. Przykładem są symulatory lotów w latach 30. ubiegłego wieku. Link trainer miał za zadanie symulować lot samolotem. W tym celu z dużą dokładnością odwzorowywał wskazania instrumentów lotniczych. W czasie II wojny światowej te oto symulatory wytrenowały ponad 500 000 pilotów.

Rys. 1.2 Link Trainer w użyciu. Źródło: [21]

Sama ta technologia tylko ilustruje to jak ludzie na wiele sposobów próbowali oszukiwać ludzi, by mieli wrażenie, że są w innym miejscu niż faktycznie są. Leży to u podstaw konceptu wirtualnej rzeczywistości. W takim razie co było pierwszym urządzeniem, które w pełni realizowało założenia wirtualnej rzeczywistości? W 1962 roku Morton Heiling zaprezentował światu Sensoramę. Można ją porównać do wyświetlania filmów w 4D. Urządzenie miało stymulować każdy zmysł człowieka. Widz mógł poczuć się jak uczestnik filmu. Przykładał on głowię do urządzenia i oglądał specjalnie nakręcone filmy. W tym czasie mógł słyszeć dźwięk w stereo, co było czymś nowym w tamtych czasach. Dodatkowo mógł poczuć różnego rodzaju zapach oraz sam fotel miał się przechylać i wibrować, by symulować zdarzenia dziejące się na filmie. Sensorama przenosiła widza w zupełnie inny świat oszukując każdy jego zmysł. Choć nadal brakowało tutaj interaktywności to ten wynalazek można określić jako pierwsze urządzenie VR.

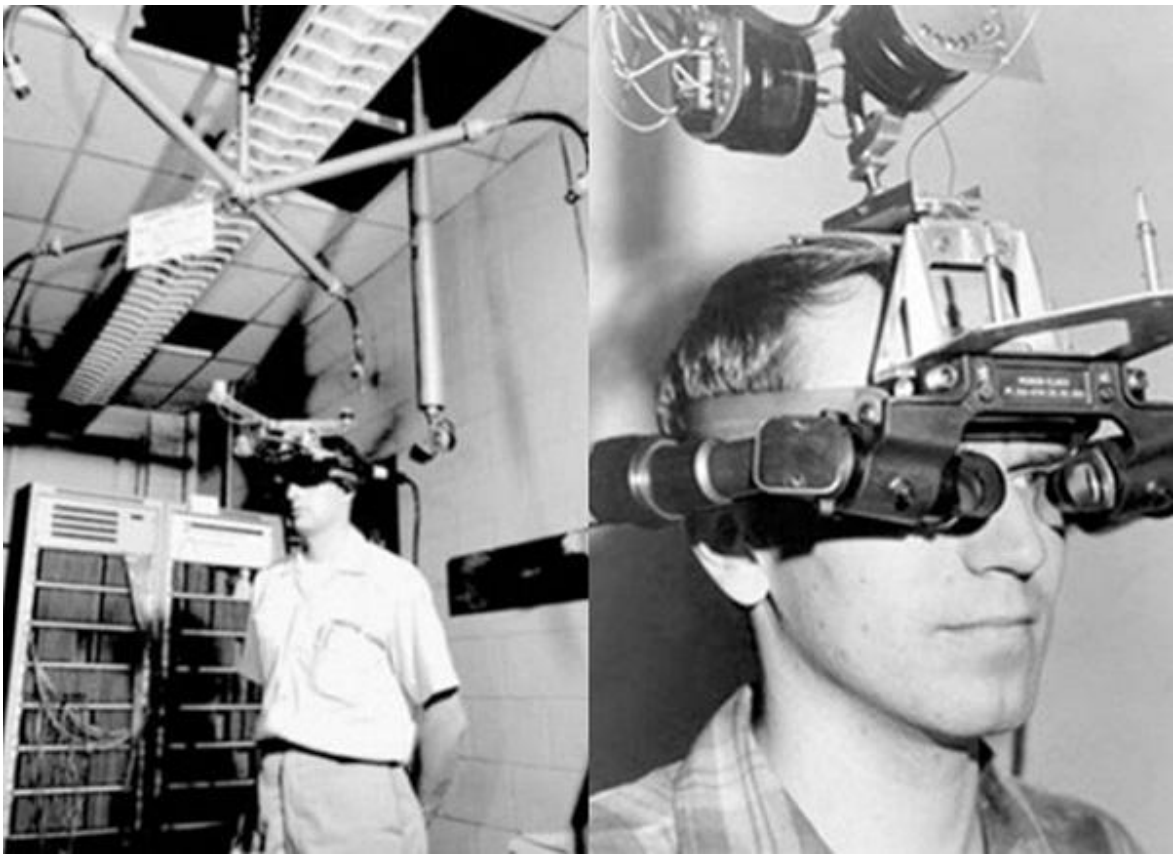
Rys 1.3 Sensorama. Źródło: [1]

Używając Sensoramy widz był nieruchomy, co jest bardzo dalekie od tego jak dziś wygląda VR. Kojarzymy często tę technologię z goglami zakładanymi na głowę. W 1961 roku firma Philco stworzyła hełm, który nie dość, że wyświetlał obraz ale jeszcze był zdolny do śledzenia ruchów użytkownika. Te informacje były przekazywane zdalnie do oddalonej kamery, której obraz był wyświetlany w hełmie. Urządzenie było stworzone, by umożliwić zdalną obserwacje niebezpiecznych miejsc. Ostatecznie hełmy były używane przez wojsko w celach treningowych. Choć nie wyświetlały one wirtualnego świata to jednak ich działanie było sporym krokiem dla VR jakie dzisiaj znamy.



Rys. 1.4 Philco Headsight. Źródło: [9]

Innym pionierem w tej dziedzinie był miecz damoklesa. W latach 60 Ivan Sutherland stworzył gogle, które nie dość że śledziły ruch głowy, ale i również wyświetlały grafikę komputerową. Gogle były przymocowane do dużego ramienia, dlatego to urządzenie żartobliwie zostało przez twórce nazwane mieczem Damoklesa, choć lepszym określeniem było Head-mounted Display, co można przetłumaczyć na ekran zamocowany na głowie. I to urządzenie można faktycznie nazwać pierwszym zestawem do VR, ponieważ użytkownik mógł wejść do wirtualnego świata i jednocześnie rozglądać się po nim. Oczywiście była to bardzo uproszczona wersja wirtualnej rzeczywistości, gdzie użytkownik nie miał większego wpływu na świat wyświetlany przed jego oczami.



Rys. 1.5 Miecz Damoklesa. Źródło: [11]

Małym przełomem w historii wirtualnej rzeczywistości były technologie opracowane w latach 80 zeszłego wieku przez firmę VPL. Nie dość, że opracowali kolejne gogle do wyświetlania grafiki komputerowej to dodatkowo zajmowali się aspektem interaktywnym wirtualnej rzeczywistości. Opracowali oni DataGlove, czyli rękawice, które śledziły ruch rąk i przesyłały te dane do komputera. W wyniku czego można było zaobserwować interakcje między człowiekiem, a obiektem na ekranie. Nawet dziś zakres działania rękawic jest imponujący, bo nie dość, że wykrywały obrót we wszystkich osiach to jeszcze miały możliwość wykrycia wygięcia palców. Ta konkretna technologia została później wykorzystana do stworzenia Power Glove przez firmę Mattel.

Rys. 1.6 Power Glove. Źródło: [13]

Rękawice były bardzo uproszczoną wersją DataGlove, bo nie wykrywał obrotu we wszystkich osiach, ale nie przeszkodziło to w byciu komercjalnym sukcesem. Niestety dobra sprzedaż nie była wyznacznikiem jakości, ponieważ osoby używające sprzętu skarżyły się na toporność w obsłudze gier, a sama ilość produkcji stworzonych na potrzeby tej rękawicy nie była spora. Power Glove pomimo fatalnej jakości stała się po części ikoną wyobrażeń o futurystycznej przyszłości. Firma VPL również opracowała strój na całe ciało, które było w stanie rejestrować ruch całego ciała. Można to porównać do strojów używanych do motion capture.

Rys. 1.7 Strój na całe ciało opracowane przez VPL. Źródło: [12]

Niestety VPL zbankrutowało pod koniec lat 80. zeszłego wieku. Powodów można się doszukiwać wszędzie, ale głównym problemem okazała się sama technologia. Była ona zbyt droga, a jednocześnie zbyt prymitywna. Ich gogle choć imponowały pomysłem to były zbyt mało wydajne. Obraz mógł wyświetlać tylko 6 klatek na sekundę, gdzie już telewizory z tamtego okresu bez problemu wyświetlały 30 klatek. To jednak nie zatrzymało postępu i w 1991 do salonów gier zawitały maszyny Virtuality. To były dość spore maszyny, które pozwalały wejść w świat wirtualnej rzeczywistości. Pierwsza wersja określona jako 1000CS oferowała gogle oraz kontrolery ruchowe. Zagrać można było w mały katalog gier przeznaczonych na wirtualną rzeczywistość. To co było wyjątkowe to możliwość grania z innymi osobami. Automaty oferowały wspólnie granie w wybranych grach. Grafika była prosta, ale można już mówić o faktycznym VR. Późniejsze wersje automatu(1000SD, 2000, 3000) oferowały lepsze podzespoły oraz lepszą rozdzielczość ekranów. Dla porównania wersja 1000CS miała w goglach dwa

ekrany LCD o rozdzielczości 372x250, gdzie wersja 2000 już miała 800x600. Widać, było że ta technologia ma duże zastosowanie w rozrywce i do takich zastosowań późniejsze wersje tej technologii będą tworzone.

Rys. 1.8 Virtuality 1000CS. Źródło: [22]

Rys. 1.9 Virtuality 2000SU. Źródło: [22]

Automaty Virtuality były bardziej nastawione na salony gier niż na zwykłego konsumenta to by wypełnić tę niszę dwie firmy postanowiły stworzyć własne gogle do wirtualnej rzeczywistości. Tymi firmami była Sega oraz Nintendo. Pierwsza firma chciała stworzyć dodatek do swojej konsoli: Sega Genesis. Miały to być gogle pozwalające zanurzyć się w cyfrowym świecie. Koszt tych gogli miał wynosić aż 200 dolarów co po uwzględnieniu inflacji wynosi aż 446 dolarów. Prototypy tych gogli były pokazywana na różnych wydarzeniach, ale ostatecznie projekt został skasowany. Powodem były problemy z chorobą lokomocyjną. I niestety ten problem dotyczy również dzisiejszych sprzętów, ale udało się zrozumieć przyczyny występowania tej choroby oraz wymyślono sposoby, by ją złagodzić. Choroba symulatorowa, bo tak jest dziś określana, występuje gdy oczy widzą ruch, a reszta ciała tego nie rejestruje. Przykładowo w grze gracz porusza

się z dużą prędkością, ale on sam w rzeczywistym świecie stoi w bez ruchu. Gracz może wtedy poczuć się jak podczas choroby lokomocyjnej. Może zacząć mu się robić słabo lub w ekstremalnych przypadkach wymiotować. Dzisiaj twórcy oprogramowania tworzą specjalne ustawienia, które mają za zadanie zmniejszyć wpływ tej choroby. Częstym środkiem zaradczym jest winieta ustawiona na ekran oraz możliwość poruszania się przy pomocy teleportacji. Jednakże największym czynnikiem, który wpływa na występowanie tej choroby to ilość klatek na sekundę. Za mała liczba zwiększa szanse na pojawienie się objawów choroby symulacyjnej. I ten powód mógł być główną przyczyną dlaczego gogle Segi zostały skasowane. Moc obliczeniowa konsoli nie mogła dać wystarczającej ilości klatek na sekundę w grach. Nintendo z drugiej strony podeszło do tematu wirtualnej rzeczywistości bardziej zachowawczo oferując bardzo prymitywną grafikę oraz bardzo ograniczoną interakcję. Zamiast gogli użytkownik dostawał stacjonarny ekran do którego trzeba było przyłożyć głowę. Grafika prezentowana przez to urządzenie była bardzo uproszczona, składające się z czerwonych diod ekrany pokazywały grafikę 3D. Lecz tylko w kolorze czerwonym. Produkt został wydany w 1995 i okazał się finansową klapą. Ludzie skarżyli się na bóle głowy, a dość niewygodny stelaż powodował, że granie w gry na tej platformie było uciążliwe.

Rys. 1.10 Gogle firmy Sega. Źródło: [23]

Rys. 1.11 Nintendo Virtual Boy. Źródło: [26]

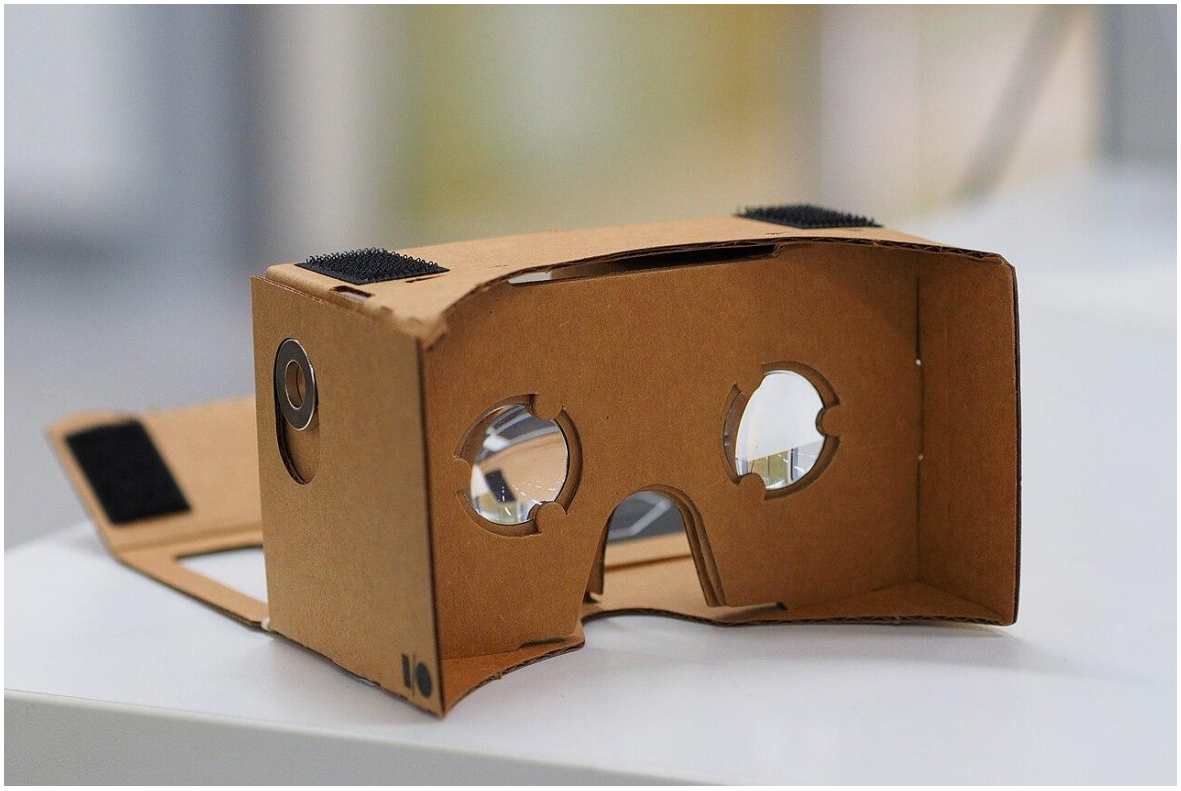
W podobnym okresie na komputery osobiste wyszły gogle VFX1, które oferowały doświadczenie zbliżone do tego co dziś się oczekuje od takich urządzeń. Miały one bardzo wysoką cenę oraz bardzo duże wymagania jak na tamte czasy, ale oferowały faktyczną interakcję użytkownika z grami. Mały kontroler, który potrafił rejestrować obrót na dwóch osiach oraz gogle, które choć miały niską rozdzielczość ekranu to można było na nich zagrać w popularne w tamtym okresie produkcje.

Rys. 1.12 Gogle VFX1. Źródło: [27]

Po tym wszystkim na kilka lat zapanowała cisza, którą przerwała prezentacja gogli oculus rift w 2012 roku. Od tego momentu zaczęła się współczesna era VR. Temat wirtualnej rzeczywistości zaczął się coraz częściej pojawiać na ustach ludzi. Firma oculus wciągu kilku lat wypuściła dwa zestawy developerskie dla twórców oraz pasjonatów. Na tym etapie gogle VR służyły głównie do oglądania specjalnie przygotowanych animacji, czy filmów, gdyż nie było możliwości jeszcze nie interakcje ze środowiskiem.

Rys. 1.13 Oculus RIft Developer Kit 1. Źródło: [28]

Wciąż największym problemem była dostępność, ale w 2014 roku Google postanowiło to zmienić. Firma uruchomiła projekt Google Cardboard, który w założeniu pozwalał każdemu posiadaczowi telefonu z androidem stworzyć własne gogle VR. Wystarczył karton, magnes, soczewki oraz telefon. Można było też kupić specjalną nakładkę na telefon, którą się zakłada na głowę i za pośrednictwem, której doświadcza się wirtualnej rzeczywistości. Bardzo małym kosztem można było zobaczyć na czym ta wirtualna rzeczywistość polega.



Rys. 1.14 Google Cardboard. Źródło: [42]

Ten rok również stał się początkiem popularności VR. Oculus przedstawił swój developer kit 2, Samsung ogłosił Samsung Gear VR, a Sony zaczęło pracę nad własnym zestawem VR. Jednakże rokiem, który można nazwać wybuchem popularności tej technologii jest 2016.

Rys. 1.15 Google Trends dotyczące hasła VR. Źródło: opracowanie własne

Nie bez powodu ludzie zaczęli się interesować tą technologią właśnie w tym roku. Wtedy premierę miał zestaw VR HTC Vive stworzony we współpracy z firmą Valve. Oferowały one pełne doświadczenie z wirtualną rzeczywistością. Użytkownik mógł nie tylko oglądać to co się dzieje w przestrzeni 3D, ale również wchodzić w interakcje z przedmiotami za pomocą kontrolerów ruchowych. To był

pierwszy takie zestaw pozwalający poruszać się w przestrzeni 3D. Wszystko było oparte o lampy. Trzeba było rozstawić w pomieszczeniu kilka specjalnych stacji, które strzelały niewidzialnym dla człowieka światłem. Za pomocą sensorów umieszczonych na samych goglach i kontrolerach komputer mógł bardzo precyzyjnie określić pozycję gracza w przestrzeni. To była rewolucja na rynku wirtualnej rzeczywistości. Dodatkowo w tym samym okresie powstała duża baza gier wykorzystujących tę technologię, więc każdy użytkownik sprzętu mógł sprawdzić jego możliwości. Problemem okazała się cena produktu oraz wysokie wymagania sprzętowe. Jednak to nie przeszkodziło w rosnącej popularności tej technologii. Z roku na rok powstawało coraz więcej filmów, gier wykorzystujących VR.

Rys. 1.15 HTC Vive. Źródło: [30]

Na kolejny kamień milowy w rozwoju nie trzeba było długo czekać, bo w 2019 roku firma Oculus wypuściła na rynek Oculus Quest oraz Oculus Rift S. Oba te zestawy do VR oferowały coś czego na co inne zestawy nie mogły sobie pozwolić. Wygodę użytkowania. Oculus Rift S oraz Oculus Quest nie posiadały specjalnych lamp, by śledzić pozycje gogli, czy kontrolerów. Do tego służyły wbudowane kamery, które śledziły pozycję kontrolerów gracza. Przez co konfiguracja sprzętu była minimalna,

bo nie trzeba było mocować specjalnych lamp do ścian. Jednakże oba sprzęty różniły się jedną bardzo podstawową funkcjonalnością. Oculus Quest w przeciwieństwie do RIft S nie wymagał komputera do poprawnego działania. To były pierwsze autonomiczne gogle VR. Miały własny system operacyjny oraz można było uruchamiać gry i programy bezpośrednio na goglach. Tak naprawdę do zabawy wystarczył sam zestaw. Można było zabrać gogle wszędzie i nie była ograniczana przez inne sprzęty. Dodatkowo miały opcje podłączenia ich do komputera, by korzystać z biblioteki gier na komputerach osobistych. Quest szybko wyparł Rift S swoją funkcjonalnością. Druga wersja Questa została jedną z najpopularniejszych zestawów do VR, oferując pełnie doświadczenia przy stosunkowo niskiej cenie.

Rys. 1.16 Oculus Quest 2. Źródło: [43]

Na dzień dzisiejszy tak się prezentuje historia wirtualnej rzeczywistości. Z każdym kolejnym rokiem technologia się rozwija i kolejne firmy tworzą własne urządzenia XR. Apple wyprodukowało Apple Vision Pro, Valve stworzyło Valve Index, a meta, dawniej oculus, dalej tworzy nowe generacje gogli VR w przystępnej cenie. Sam rynek pozostaje niszą, ale coraz więcej ludzi jest w stanie sobie pozwolić na doświadczenie wirtualnej rzeczywistości, czy nawet rozszerzonej, gdyż każde kolejne generacje gogli oferują AR oraz MR. Ta konkretna technologia przeszła

sporo zmian i dziś jesteśmy w momencie, gdzie taki wynalazek nie jest tylko czymś z książek science fiction, a faktycznie realnym urządzeniem.

Rys. 1.17 Apple Vision Pro. Źródło: [37]

1.3 Zastosowanie

Wirtualna rzeczywistość posiada coś czego inne media nie mogą zaoferować. Wejście w świat cyfrowy. Nie ma tu pośredników, czy rozpraszaczy. Użytkownik jest w tym świecie i może swobodnie się po nim poruszać. Doświadczenie jest bardziej naturalne, gdyż zamiast klikać na klawiaturze, czy kontrolerze, by otrzymać akcje to użytkownik fizycznie ~~to robi~~. Może podnieść przedmiot schylając się lub zobaczyć coś po prostu kierując swoją głowę odpowiednią stronę. Przy projektowaniu wnętrz zastosowanie wirtualnej rzeczywistości jest pomysłem dość oczywistym. Za pomocą VR użytkownik może w pełni zobaczyć skalę i poczuć się jak w gotowym pomieszczeniu. Coś co z lotu ptaka wygląda dobrze, może nie wyglądać z pierwszej osoby. Tu VR pomaga w zobaczeniu jak w rzeczywistości zaprojektowane wnętrze może wyglądać. Wcześniej zaprezentowane programy w niektórych przypadkach pozwalały zobaczyć gotowe pomieszczenie przy użyciu zestawu VR. Niestety takie rozwiązania są tylko w celach obserwacyjnych, a możliwa interaktywność jest ignorowana. Wirtualna rzeczywistość jest świetna w wizualizowaniu projektów, ale też może być użyteczna w jej projektowaniu, oferując bardziej naturalny i intuicyjny sposób tworzenia. Technologia VR jest dziś na takim poziomie, że projektowanie wnętrz powinno być przystępniejsze dzięki niej. Takie rozwiązanie mogłoby mieć zastosowanie dla firm meblarskich, gdzie kupujący mogą sami zaprojektować pokój bez większego nakładu pracy. Biorąc pod uwagę jak VR jest naturalny w swojej obsłudze każdy mógłby szybko stworzyć wymarzony pokój. Wirtualna

rzeczywistość nie jest ograniczona tym, gdzie jest, więc nie ma problemu w stworzeniu pomieszczenia, który by przypominał rzeczywisty odpowiednik. Pozwala to na używanie takiego rozwiązania wszędzie. Szczególnie, że dzisiejsza technologia VR nie wymaga dodatkowego komputera do uruchamiania programów. ~~Tworząc~~ samo rozwiązanie jeszcze bardziej przystępniejszym. Ta technologia jeszcze bardziej pozwoli na stworzenie dokładnych wizualizacji bez godzin nauki programów, ponieważ wszystkie akcje będą wykonywane naturalnie i intuicyjnie. Taki system dekoracji wnętrz byłby idealnym oprogramowaniem dla dużych firm meblarskich lub nawet dla osoby prywatnej. Łatwiej jest coś zrobić fizycznie niż przeklikując sporą ilość zakładek, by umieścić jedną rzecz w projekcie.

2. Część Teoretyczna

Celem pracy jest stworzenie systemu do projektowania wnętrz wykorzystując do tego wirtualną rzeczywistość. Cały system ma zostać stworzony w oparciu o Unreal Engine 5 z wykorzystaniem podejścia no code.

2.1 Dziedzina Pracy

(W sumie nie wiem co tu napisać)

2.2 Dobór Technologii VR

Głównym założeniem projektu jest opracowanie systemu, dzięki któremu użytkownik będzie wstanie zaprojektować i zwizualizować pomieszczenie wykorzystując do tego zalety wirtualnej rzeczywistości. Aktualnie na rynku nie ma podobnego systemu, który by wykorzystywał VR do procesu projektowania. Jedynie są one w stanie zrobić wirtualny pokaz gotowego pomieszczenia. Ten projekt pozwoli na samodzielne stworzenie i udekorowanie pokoju w wirtualnej przestrzeni.

Pierwszym problemem jest wartość wirtualnej rzeczywistości i dlaczego to ona powinna być podstawą systemu. Aktualnie można wyróżnić trzy ~~technologię~~, które oferują różny sposób doświadczania treści. Flat, AR i VR. Pojęcie Flat odnosi się do ekranów monitora. Wszystkie informacje są przekazywane przez ekran monitora i użytkownik kontroluje je za pomocą myszki i klawiatury. To jest standardowe rozwiązanie, pozwalające na stworzenie bardzo rozbudowanych programów kosztem prostoty ich obsługi i czytelności. Większość programów do projektowania wnętrz opiera się o płaski ekran monitora. Takich rozwiązań jest za dużo i tworzenie kolejnego nie miałoby ~~sensu~~. Z drugiej strony jest technologia

rozszerzonej rzeczywistości, polegająca na nakładaniu na rzeczywisty obraz cyfrowych obiektów. Na pierwszy rzut oka wydaję się to idealne na taki projekt. Jednakże to rozwiązanie ma jeden zasadniczy problem. Wymagany jest rzeczywisty obraz. AR sprawdziłoby się idealnie na puste pomieszczenie, ale rzadko takie warunki mogą się zdarzyć. Przykładowo podczas planowania remontu, stare umeblowanie nadal istnieje, co przeszkadzałoby w dokładnym zaprojektowaniu nowego pomieszczenia. Innym problemem jest wymóg bycia w danym pokoju. Trzeba zaprojektować dane pomieszczenie tu i teraz, nie można tego robić z innego miejsca, bo wymagany jest rzeczywisty obraz pokoju. Co może być zaletą jeśli warunki są idealne w przeciwnym wypadku AR, nie spełnia wymagań do stworzenia tego projektu. Aktualna technologia również nie kładzie dobrego światła na AR. Choć dziś jest sporo urządzeń pozwalających na wykorzystanie rozszerzonej rzeczywistości to nadal sama technologia potrzebuje dopracowania. Telefony są za bardzo ograniczające ze względu na swój rozmiar, a nawet gogle Meta Quest 3, które oferują technologię AR to jest ona tam bardzo niedopracowana. Jedynie Apple Vision Pro miałoby zastosowanie, ale cena, ograniczona interaktywność i hermetyczność systemu nie pozwalałaby na stworzenie funkcjonalnego systemu. Trzecią opcją jest technologia VR. Dzisiejsza technologia pozwala na wygodne i bezproblemowe korzystanie z wirtualnej rzeczywistości. Umożliwia tworzenie wirtualnego świata niezależnie od aktualnego położenia użytkownika. Dodatkowo uwzględnienie kontrolerów ruchowych pozwala na naturalny poziom interakcji z otoczeniem. Można powiedzieć, że łączy ona zalety Flat i AR. Jest kompromisem wobec tych dwóch rozwiązań. Nie można zbudować bardzo skomplikowanego systemu na poziomie tych tworzonych na Flat kosztem intuicyjności, ponieważ cały sens VR polega na naturalności interakcji i obserwacji. Natomiast można stworzyć system przystępny dla nowego użytkownika, który bez znajomości skomplikowanego interfejsu będzie wstanie zaprojektować wnętrze według swoich upodobań oferując nadal swobodę w tworzeniu i sporą funkcjonalność.

Wybranie samej technologii to jedna rzecz, ale wybranie sprzętu, który uruchomi projekt to inna sprawa. Aktualnie na rynku jest spora liczba urządzeń VR i wybranie jednego konkretnego jest wyzwaniem, w szczególności, że potrafią znacząco różnić się od siebie. Warto spojrzeć na ich możliwości oraz co będzie ważnym atutem dla projektu.

Tabela 2.1 Porównanie popularnych zestawów XR

Nazwa	Typ	Cena	Rozdzielczość na oko	Odświerzanie	CPU	Kontrolery	Bateria
Meta Quest 3	Autonomiczne	$499	2064x2208	120 Hz	Octa-core Kryo (1 x 3.19 GHz, 4 x 2.8 GHz, 3 x 2.0 GHz)	2 x Meta Quest Touch Plus Controllers	2.2 h
~~Meta Quest 3S~~	Autonomiczne	$299	1832x1920	120 Hz	Octa-core Kryo (1 x 3.19 GHz, 4 x 2.8 GHz, 3 x 2.0 GHz)	2 x Meta Quest Touch Plus Controllers	2.5 h
Samsung Galaxy XR	Autonomiczne	$2048	3552x3840	90 Hz	Snapdragon XR2+ Gen 2	2 x Samsung Galaxy XR Controller	2 h
Valve Index	Wymagają PC	$999	1440x1600	144 Hz		2 x Index Controller	
Apple Vision Pro M5	Autonomiczne	$3499	3660x3200	120 Hz	10-core (4 x 4.6 GHz, 6 x 3.0 GHz)		2.5 h
HTC Vive Pro 2	Wymagają PC	$1399	2448x2448	120 Hz		2 x Vive Pro Controller	
Pico G3	Autonomiczne	$399	2832x1920	90 Hz	Octa-core Kryo 585 (1 x 2.84 GHz, 3 x 2.42 GHz, 4 x 1.8 GHz)	1 x Pico G3 Controller	2.5 h
Playstation VR 2	Wymagany PC	$399	2000X2040	120 Hz		2 x Playstation VR2 Sense Controlle	

Pierwszą rzeczą jaką należy porównać to typ urządzenia. Autonomiczne mają tę zaletę, że nie trzeba dodatkowego komputera do obsługi programów. Można uruchamiać projekty bezpośrednio na goglach. Jest to wygodniejsze rozwiązanie, bo nie trzeba się martwić okablowaniem, raz jest tańsze, bo wystarczy zakup jednego sprzętu, a cena komputera nie jest liczona w cenę całej konfiguracji. Nie zapominając, że większość gogli autonomicznych oferuje połączenie z komputerem za pomocą kabla lub po wifi. Jednakże gogle wymagające komputera oferują w zamian większą funkcjonalność. Dobrze będzie się najpierw nim przyjrzeć. Valve Index są wypełnione mniejszą i większą funkcjonalnością. Tu na pierwszy plan wysuwają się kontrolery, które zamiast być pilotami, bardziej służą jako nakładka na dłonie. Mają dużą liczbę sensorów, które wykrywają położenie palców oraz nacisk na kontroler. Dając użytkownikowi jeszcze większą możliwość interakcji w świecie. W dodatku pozycja gogli jak i kontrolerów jest śledzona bardzo precyzyjnie przy użyciu specjalnych lamp, więc nie można mówić o gubieniu przez zestaw pozycji kontrolerów. Duża częstotliwość odświeżania ekranów dodatkowo wpływa na komfort użytkownika niwelując chorobę symulatorową. Wadą będzie cena i dostępność. Koszt prawie tysiąca dolarów na sprzęt, który jeszcze wymaga komputera do obsługi jest absurdalnie duży. Ten zestaw VR wyszedł w 2019 roku i dostępność poszczególnych akcesorii jest utrudniony. Można używać zamienników i firma valve nawet to zaleca, ale kupno całego zestawu na oficjalnej stronie nie jest już możliwe. Te gogle biorą udział w tym porównaniu głównie ze względu na ich funkcjonalność. Nawet dziś są jedynymi w swoim rodzaju. Podobnie można opisać HTC Vive Pro 2, tylko że pod wieloma względami są gorsze od sprzętu Valve. Nie poprawia sytuacji cena, która jest jeszcze większa. Ekrany w goglach mają większą rozdzielczość, ale mają mniejsze odświeżanie. 120 Hz to nie jest zły wynik i jest standardem wśród budżetowych gogli, ale to jest sprzęt za 1399 dolarów. Kontrolery również się nie wyróżniają na tle konkurencji. Są one wielkie i nie poręczne. Ten zestaw jest za drogi i nie oferuje lepszej wydajności. Ostatni z tej kategorii jest playstation VR 2. Choć mylnie można założyć, że do obsługi jest potrzebna konsola playstation 5 to firma Sony udostępniła przejściówki pozwalające na podłączenie ich sprzętu do komputera. W przeciwieństwie do rozwiązań konkurencji te gogle nie wymagają

specjalnych lamp. Playstation VR 2 śledzi pozycje kontrolerów przy pomocy wbudowanych kamer. Jest to o wiele wygodniejszy sposób, ponieważ podłączenie i ustawienie całej konfiguracji zajmuje mniej czasu. Problem pojawia się przy precyzji śledzenia. Nie jest ona tak dokładna jak przy użyciu lamp, a kamery mają trudność ze śledzeniem kontrolerów, gdy te są poza ich polem widzenia. W takich sytuacjach można powiedzieć, że gogle zgubiły kontrolery. Jedyne informacje jakie są interpretowane to te przez żyroskop kontrolera. Nie jest to duży problem, gdy system jest zaprojektowany, żeby większość uwagi użytkownika była przed nim w polu widzenia kamer. Sprzęt Sony również ma funkcję śledzenia oczu, czy kontrolery posiadają adaptatywne spusty. Ta pierwsza funkcja jest na tyle kluczowa, bo pomaga przy optymalizacji programów, gdzie tylko ta przestrzeń na którą użytkownik patrzy będzie renderowana w wysokiej rozdzielczości. Niestety ta funkcjonalność nie jest wspierana na komputerach, jak i również inne bardziej zaawansowane funkcje. Czyni to ten zestaw bardzo wybrakowanym, bo choć ma dużo funkcji to nie można ich użyć. Przynajmniej cena jest o wiele niższa niż u konkurencji, czyniąc je najbardziej opłacalnymi. Zaletą jaką te trzy zestawy mają to brak baterii. Gogle działają tak długo jak działa komputer. Nie trzeba ich dodatkowo ładować. Z tych trzech zestawów wybrałbym do projektu playstation VR 2 głównie przez swoją cenę, ale większość autonomicznych gogli oferuje większą elastyczność, funkcjonalność przy często niższej cenie.

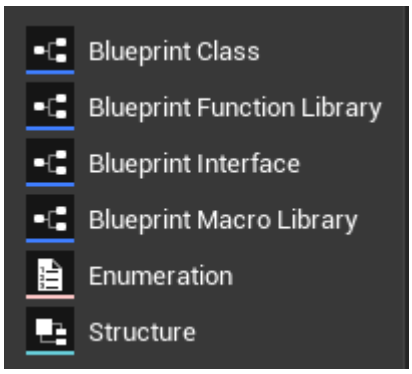
Najbardziej popularnymi goglami autonomicznymi są flagowe zestawy Meta Quest. Do porównań użyje tych najbardziej aktualnych, czyli 3 i 3S. Oparte o procesor Octa-core Kryo potrafią uruchamiać sporą gamę programów wykorzystujących grafikę 3D. W większości nie jest to bardzo szczegółowa grafika, ale jest na tyle zaawansowana, że może stać obok komputerów, czy konsol. Mocą i możliwością można porównać do konsoli Playstation 3, czy Xbox 360. Odświeżanie i rozdzielczość ekranów jest naprawdę duża. 120 Hz przy rozdzielczości 2064x2208 jest imponujące. To oczywiście jest statystyka dla podstawowej wersji Meta Quest 3. 3S ma niższą rozdzielczość, ale reszta parametrów jest taka sama. Kontrolery, choć nie mają sporej funkcjonalności to oferują tą najbardziej podstawową i wymaganą. To co jest ważnym czynnikiem to cena. 499 dolarów za podstawową wersje i 299 dolarów za 3S jest rozsądne przy tym jak mocne są gogle oraz ile oferują możliwości. System jest oparty o Androida, więc budowanie aplikacji nie jest bardzo skomplikowane, dzięki sporej

ilości dokumentacji. Tańszą alternatywą dla Meta Quest są Pico G3. Jego parametry są niższe. Rozdzielczość jest minimalnie mniejsza, a odświeżanie wynosi tylko 90 Hz. Mniejszą częstotliwość ma też procesor, choć tej samej marki to jego parametry są również minimalnie mniejsze. Główną wadą jest tylko jeden kontroler. Jest to poważny problem, gdyż przy tworzeniu aplikacji VR to zakłada się, że użytkownik będzie posiadał dwa kontrolery. Jeden na lewą rękę, drugi na prawą. W tym zestawie VR tego nie ma, ale jest to kluczowe przy projektowaniu systemu. Nie pomaga wysokość ceny, bo ten zestaw kosztuje 399 dolarów, a za mniejszą cenę można kupić gogle Meta Quest 3S, które posiadają oba kontrolery. Pico G3 wydaję się produktem bardziej przeznaczonym do biznesu oraz obsługi AR, gdzie jeden kontroler w zupełności wystarcza. Innym bardziej biznesowym rozwiązaniem są gogle Apple Vision Pro. Jest to produkt premium, który już ze swoją ceną przegrywa z resztą urządzeń. Oferuję sporą moc obliczeniową oraz wysoką rozdzielczość ekranów, ale Apple nie oferują kontrolerów. Zamiast tego jest śledzony ruch rąk. Taką funkcjonalność mają również gogle Meta Quest, ale produkt Apple ma to śledzenie bardziej dopracowane. Cała interakcja opiera się na gestykulacji użytkownika. Zbudowanie systemu opartego o gesty użytkownika jest możliwe, ale wymaga zbyt dużej pracy jak na jedną osobę. Ten produkt jest bardziej przeznaczony do rozwiązań AR, gdzie prosta gestykulacja ma większe zastosowanie. W VR taki sposób sterowania nie miałby dużego zastosowanie przez brak precyzji. To samo można powiedzieć o tańszej alternatywie dla sprzętu Apple'a, czyli Samsung Gear XR. W przeciwieństwie do swojego konkurenta Samsung do swojego zestawu załącza kontrolery. Niestety cena nadal jest wielokrotnie większa od sprzętu Meta. Duża rozdzielczość, czy moc obliczeniowa nie usprawiedliwia tak dużej ceny. Ostatnim parametrem, który ma duży wpływ jest bateria. Każde z gogli autonomicznych trzeba ładować. I dla każdego wcześniej wymienionych czas działania na baterii średnio oscyluje wokół dwóch godzin. Jest to mocno niedokładne, bo to ile gogle zużyją baterii ściśle jest związane z zużyciem mocy operacyjnej. Bardziej wymagające aplikacje zwiększą zużycie baterii. I to jest największy problem zestawów autonomicznych. Po sesji użytkowania trzeba ładować gogle, co nie jest zbyt optymalnym rozwiązaniem. Nawet podłączenie do komputera, czy ładowarki nie powoduje, że gogle będą działać dłużej. Na potrzeby tego projektu po analizie zalet i wad różnej gamy sprzętów ~~zdecydowa~~łem, że zestaw VR Meta Quest 3 będzie najbardziej

odpowiednim wyborem. Są one dobrym kompromisem między wydajnością, funkcjonalnością, a ceną. Pomaga w tym również fakt istnienia gotowych rozwiązań dla tej platformy. Dlatego ły system będzie domyślnie działać na Meta Quest 3.

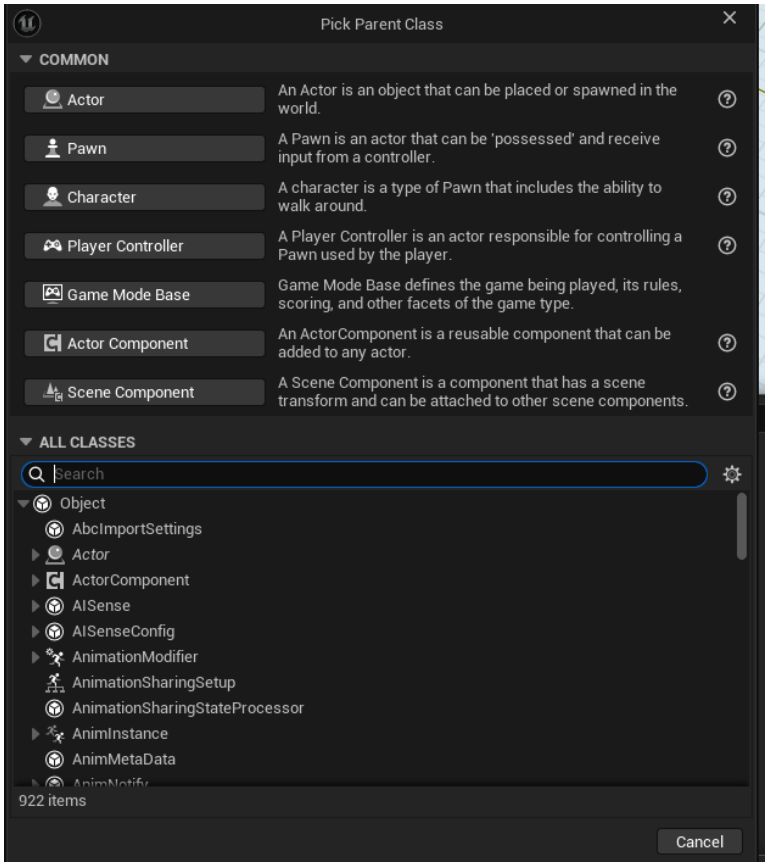
2.3 Unreal Engine 5

Ważnym elementem projektu jest środowisko w jakim będzie system projektowany. ~~Wybrałem~~ do tego Unreal Engine 5, ~~ponieważ byłem już z nim~~ zaznajomiony oraz oferuje on spore wsparcie dla technologii wirtualnej rzeczywistości. Ten silnik graficzny oferuje darmową licencję dla osób prywatnych, których przychód brutto nie przekracza miliona dolarów, małych firm, których roczny przychód brutto nie przekracza miliona dolarów oraz dla celów naukowych. W te ostatnie kryteria ten projekt się wpasowuje. Unreal Engine 5 to piąta iteracja silnika graficznego Unreal Engine wypuszczony przez Epic Games Inc. Jest on jednym z najpopularniejszych środowisk do tworzenie gier wideo, ale i również tworzenia wizualizacji. Jednym z głównych cech jest możliwość tworzenia zaawansowanej grafiki, bez większej znajomości programowania. Natomiast jest możliwe projektowanie systemów z wykorzystaniem języka C++. Jednakże w tym projekcie postanowiłem zastosować podejście no code. To konkretne podejście umożliwia tworzenie całych aplikacji bez znajomości programowania. Wszystko polega tworzeniu logiki systemu za pomocą wcześniej przygotowanych bloków. Unreal Engine 5 wspiera oraz zaleca takie podejście. Cały proces tworzenia aplikacji u podstaw stoi na koncepcie blueprintów. Można je porównać do klas w klasycznym programowaniu.



Rys. 2.1 Przykładowe typy blueprint. Źródło: opracowanie własne

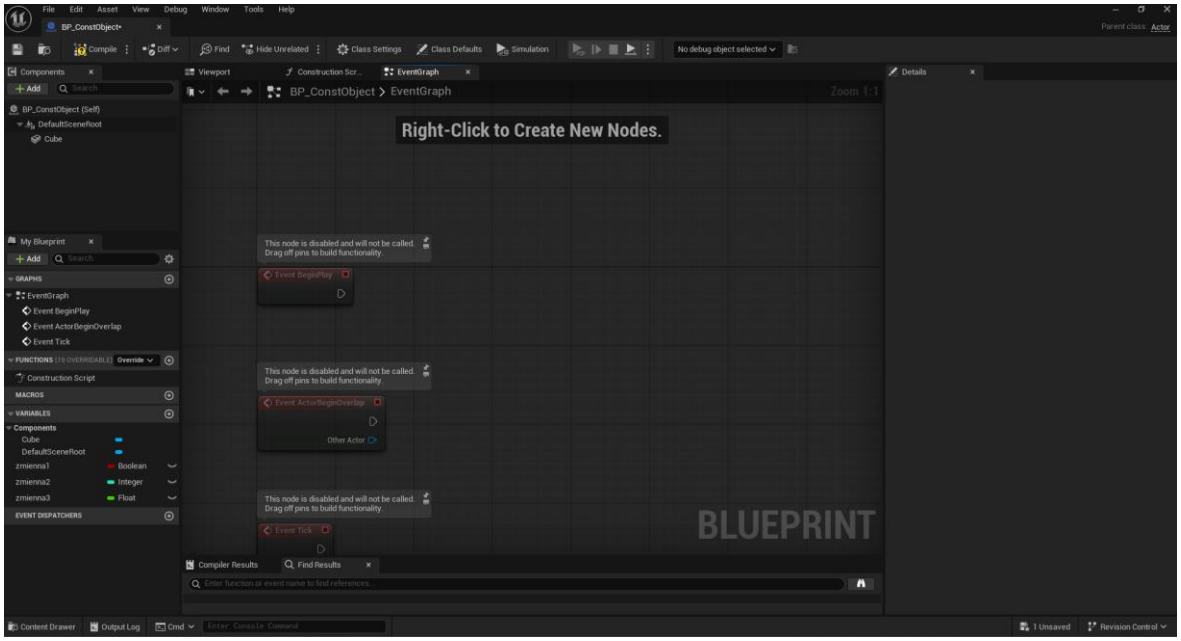
Blueprinty reprezentują pojedyncze aspekty systemu lub jeden obiekt w przestrzeni 3D. W zależności od jakiej klasy blueprint dziedziczy, taka będzie jego funkcjonalność.



Rys. 2.2 Tworzenie blueprint. Źródło: opracowanie własne

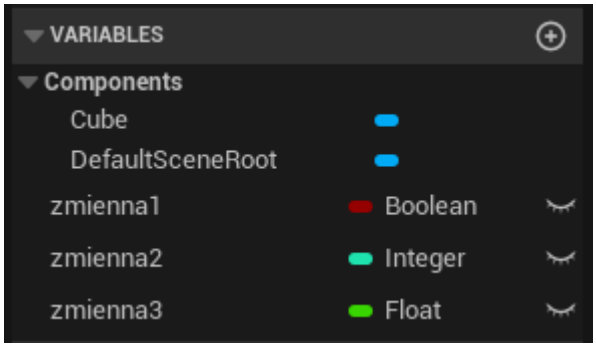
Na potrzeby tego projektu można wyróżnić trzy podstawowe klasy dziedziczne: Actor, Pawn oraz Actor Component. Blueprinty typu actor to obiekty, które można postawić w przestrzeni 3D. Typ Pawn jest obiektem sterowanym przez użytkownika. Wszystkie jego reakcje są interpretowane przez typ Pawn, co ważne, że typ Pawn sam dziedziczy od typu actor. Actor Component nie jest samodzielnym obiektem, a raczej można go porównać do interfejsu. Można go przyczepić do dowolnego obiektu i w ten sposób wzbogacić funkcjonalność blueprintu typu actor. Wartym wspomnienia typem jest Game Mode Base. W teorii on powinien zawierać całą logikę projektu oraz charakteryzować sposób użytkowania. Jednakże domyślne ustawienia wystarczają na potrzeby projektu. Tak jak było wspomniane blueprint można porównać do tradycyjnej klasy. Można tworzyć zmienne określonego typu: Int, Float, String, ale też innych blueprintów.

Każdy blueprint zawiera w sobie odpowiednik funkcji main. W edytorze nazywa się to Event Graph.



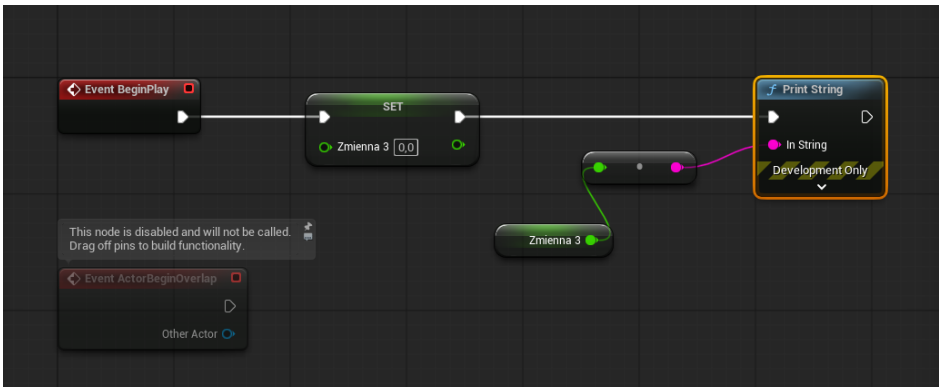
Rys. 2.3 Edytor Blueprint. Źródło: opracowanie własne

Logikę systemu tworzy się za pomocą bloków o konkretnej funkcjonalności. Bloki reprezentują klasyczną funkcjonalność tradycyjnego kodu. Na Main Graphie widać trzy bloki eventowe. Są one odpowiedzialne za uruchamianie kodu w konkretnej sytuacji. Domyślnie są trzy. Konstruktor, Wykrycie kolizji oraz funkcja uruchamiana co każdy tick, czyli co określony czas. Można też tworzyć własne bloki eventowe, które mogą służyć jako punkty końcowe dla innych aktorów. W głównej mierze jednak to są funkcje. Tak jak w tradycyjnej klasie można tworzyć własne zmienne o określonym typie.



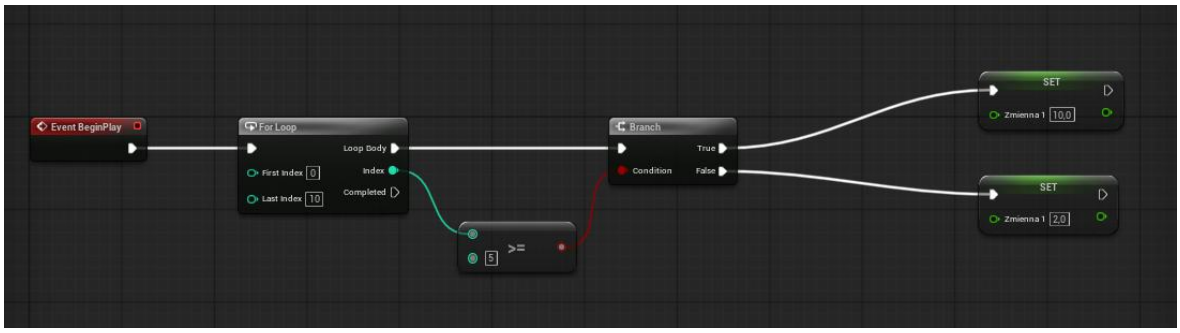
Rys. 2.4 zmienne. Źródło: opracowanie własne

Zmienne mogą być jednym z prymitywnych typów lub być odwołanie do innego blueprinta. Można do nich przypisać wartość domyślną oraz ustawić jaką formą zmiennej ma być: pojedyncza, tablica, mapa, zbiór. Do zmiennej można przypisać wartość lub ją odczytać.



Rys. 2.5 Przypisanie i odczytanie zmiennej. Źródło: opracowanie własne

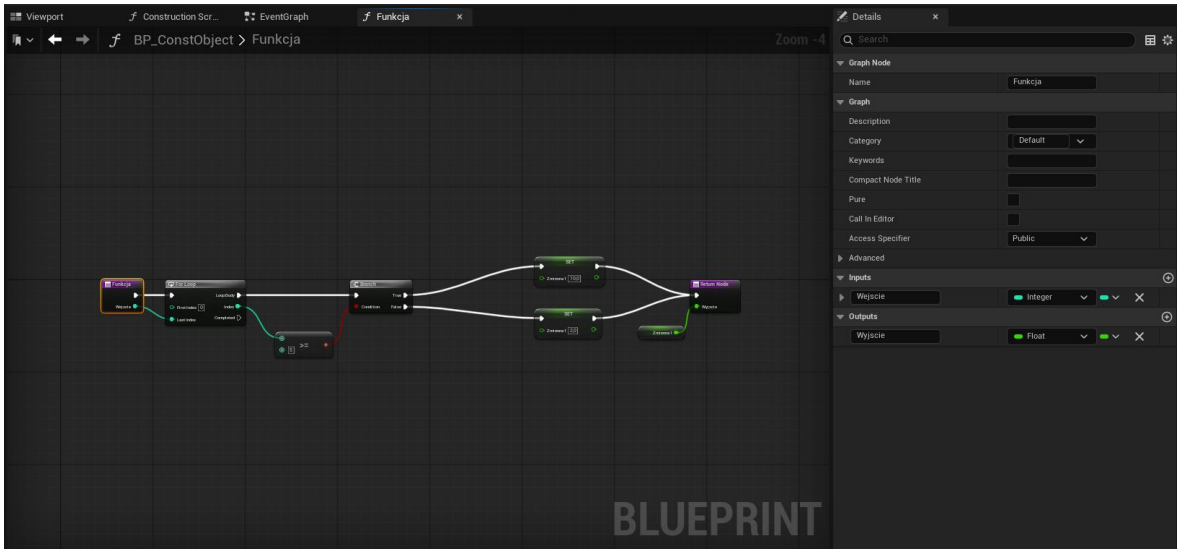
Cały proces tworzenia programu opiera się na łączeniu bloków w odpowiedniej kolejności. Do jednego bloku może prowadzić wiele ścieżek, ale z jednego bloku może prowadzić jedno wyjście. Biblioteka bloków jest ogromna i obejmuje proste operacje arytmetyczne, logiczne, czy też bardziej zaawansowane operacje. Można też tworzyć pętle, czy rozgałęzienia, które są odpowiednikiem wyrażenia if.



Rys. 2.6 Stworzenie pętli oraz rozgałęzienie. Źródło: opracowanie własne

Ważnym elementem tworzenia struktury kodu są parametry oraz zwracane wartości. Mogą być dowolnego typu i za pomocą nich można stworzyć wielofunkcyjny kod. Parametr może przyjąć wartość wpisaną lub być wartością pobraną ze zmiennej. Wtedy trzeba stworzyć blok, który zwraca wartość zmiennej i go przyczepić do wejścia parametru. Blok może zwracać kilka wartości

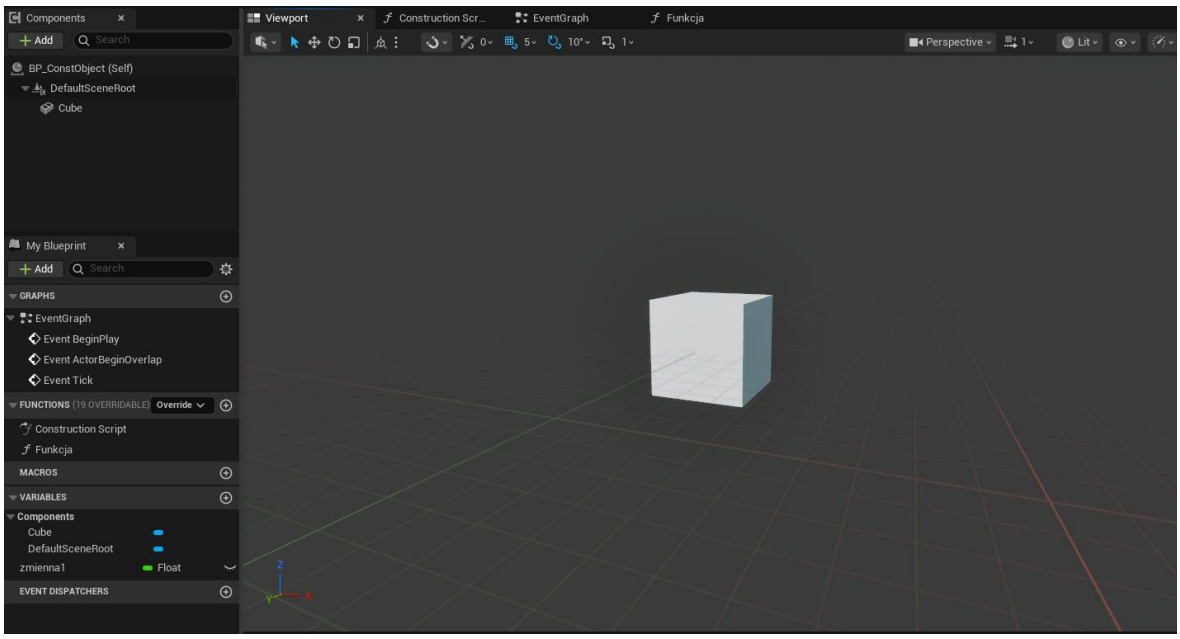
jednocześnie. Te wartości można wykorzystać później w programie. Poza używaniem wbudowanych bloków funkcji można też tworzyć własne.



Rys. 2.7 Stworzenie funkcji. Źródło: opracowanie własne

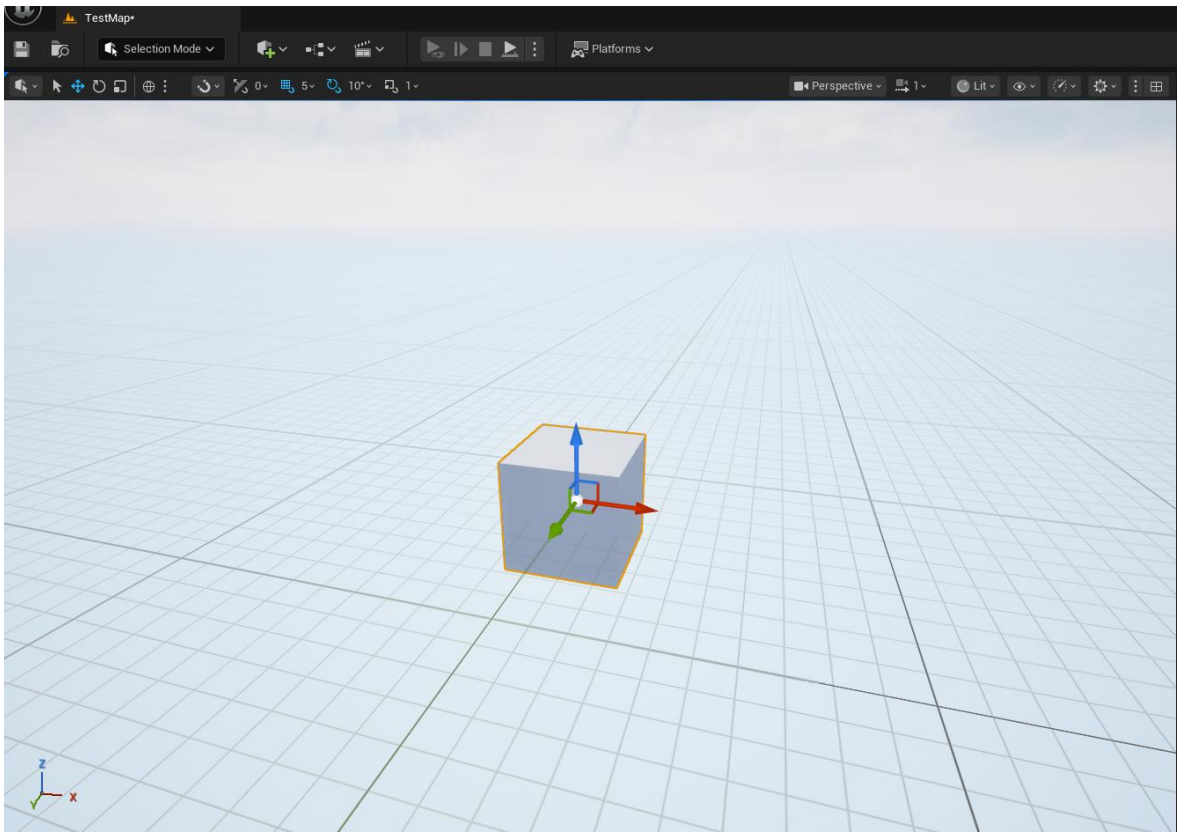
Stworzenie funkcji tworzy nową zakładkę w edytorze. Można określić liczbę parametrów wejściowych oraz wyjściowych. Ciało funkcji składa się z bloków. Nie różni się to od tworzenia logiki w zakładce event graph, ale pozwala na stworzenia bardziej modularnego i czytelniejszego kodu. Stworzona funkcja może być wywołana w każdym momencie w obrębie blueprinta.

Blueprint to nie tylko tworzenie kodu, ale też tworzenie obiektu reprezentującego logikę. Sposób jego reprezentacji zależy od typu blueprinta. Najbardziej podstawy oferuje stworzenie obiektu składającego się z brył, oświetlenia, źródeł dźwięku, czy innych struktur. Każdy obiekt reprezentujący może być modyfikowany wewnątrz kodu.



Rys. 2.8 Obiekt reprezentujący blueprint. Źródło: opracowanie własne

By obiekt był aktywny i można go było używać trzeba go postawić na mapie w edytorze Unreal Engine.



Rys. 2.9 Obiekt w edytorze mapy. Źródło: opracowanie własne

To są tylko podstawy używania blueprintów w Unreal Engine 5, ale ich zastosowanie jest o wiele większe. Na podstawie ich można tworzyć zaawansowane i wielofunkcyjne systemy. Są one przydatne przy prototypowaniu funkcjonalności, bo pozwalają w małym odstępie czasowym stworzyć coś, co ma działać zgodnie z wymaganiami. Samo podejście no code w Unreal Engine 5 nie tylko ogranicza się do tworzenia logiki, bo jest integralną częścią każdego aspektu tworzenia projektu. Za pomocą ich można stworzyć zaawansowane materiały, które można nałożyć na obiekty 3D. Nie trzeba znać żadnego języka programowania, by zaprojektować całe systemy. Jednakże podejście no code, choć jest wygodne w użyciu i szybko pozwala tworzyć logikę programu to ma swoje wady. Główną jest jej słaba czytelność przy bardziej skomplikowanych instrukcjach. Im bardziej zawansowana funkcjonalność, tym mniej czytelny jest kod. Każdy blok ma swój rozmiar i tworzenie kolejnych funkcji zabiera miejsce w edytorze. Przy kilku małych systemach ten problem jest nie widoczny, ale gdy tylko jest potrzeba zaimplementować bardziej zaawansowaną funkcjonalność to ilość potrzebnych bloków i połączeń sprawia, że kod jest nieczytelny. Można to zaradzić tworzeniem kolejnych funkcji pomocniczych, które zawierają pomniejszy kod, ale późniejsze odczytanie kodu jest utrudnione przez sam edytor. Można też wspomnieć, że kod stworzony za pomocą blueprintów nie jest tak samo wydajny jak stworzony za pomocą C++. Choć u podstaw leży ten język programowania to jednak nie mamy kontroli nad nim, gdy tworzymy w no code. Możliwości optymalizacyjne są bardzo ograniczone. Podsumowując, Unreal Engine 5 umożliwia tworzenia programów z wykorzystaniem podejścia no code, choć jest wygodny i pozwala na szybkie prototypowanie systemów to jednak ma ograniczenia jeśli chodzi o optymalizacje oraz czytelność kodu.

2.4 Założenia i Funkcjonalność aplikacji

Aplikacja u podstaw powinna umożliwiać zaprojektowanie jednego pomieszczenia wykorzystując technologię wirtualnej rzeczywistości. Projekt ma być intuicyjny w użytkowaniu oraz być przystępny dla osób, które nie miały styczności z wirtualną rzeczywistością.

Pierwszą ważna funkcjonalnością powinno być zastosowanie technologii wirtualnej rzeczywistości. Na to składa się: poruszanie się w przestrzeni trójwymiarowej, śledzenie ruchu dłoni przy pomocy kontrolerów, interakcja z obiektami w przestrzeni trójwymiarowej oraz natywne uruchamiane projektu na

zestawie VR. To jest podstawa całego projektu i wszystkie inne funkcjonalności powinny opierać się na fakcie, że użytkownik korzysta z technologii VR. Jako że do projektu wybrałem zestaw Meta Quest 3 to mogę stworzyć projekt i uruchomić go natywnie na tym zestawie. Dzięki temu mogę uniknąć pośrednika jakim jest komputer oraz ułatwić dostęp do użytkowania aplikacji. Forma poruszania się w przestrzeni 3D jest funkcją dość skomplikowaną, ponieważ istnieje wiele form w jakich użytkownik może się przemieszczać. Najbardziej popularnymi jest teleportacja oraz płynne poruszanie się. Teleportacja jest dobra dla osób z chorobą symulacyjną, ale jednocześnie może powodować trudności z orientacją w przestrzeni. Płynne poruszanie z drugiej strony, choć jest mniej przyjemne dla osób z chorobą symulacyjną to zwiększa imersje użytkownika, daje mu lepsze spojrzenie na przestrzeń 3D. Sam projekt nie będzie zawierać elementów, które by pogarszały komfort użytkowania, więc płynne poruszanie będzie podstawą do przemieszczania się po przestrzeni trójwymiarowej. Interakcja z przedmiotami powinna opierać się na podstawie kontrolerów śledzących ruch dłoni użytkownika. Osoba korzystająca z aplikacji powinna móc naturalnie wyciągnąć rękę i podnieść dany przedmiot. Dodatkowo wirtualne dłonie powinny być fizycznymi obiektami, czyli powinny wchodzić w kolizje z innymi obiektami oraz blokować się na obiektach. Cała interakcja w środowisku trójwymiarowym powinna być intuicyjna i prosta dzięki zastosowaniu tej technologii.

Drugim ważnym filarem tego projektu jest sam system projektowania wnętrz. Podstawową funkcjonalnością jest generowanie jednego pomieszczenia. Projekt będzie miał małą skalę, dlatego będzie możliwość generacji tylko jednego pokoju. Samo tworzenie pomieszczenia powinno być proste i wymagać jak najmniejszej ilości informacji od użytkownika. Po stworzeniu pomieszczenia użytkownik powinien miał dowolność w kreowaniu jego wyglądu. Głównym narzędziem do tego zadania będzie tablet. Wszystkie czynności związane z funkcjonalnością będą dostępne za pośrednictwem tabletu. Obsługa powinna być prosta i oparta na dotyku. Tablet ma działać i funkcjonować tak jak prawdziwy odpowiednik. Jedną z funkcjonalności powinna być możliwość resetu całego pomieszczenia, by umożliwić zmianę wymiarów pomieszczenia. Reszta funkcjonalności dzieli się na trzy kategorię. Tworzenie przedmiotów, zmiana wyglądu ścian oraz tworzenie stałych obiektów.

Przedmioty stworzone przez użytkownika mogą być podniesione, rzucone, a ich zachowanie jest oparte o fizykę. Łączy się z nim też inna funkcjonalność, bo by ułatwić projektowanie pomieszczenia i zapobiec przypadkowym zmianom lokacji przedmiotów, użytkownik, będzie miał możliwość zamrażania przedmiotów. Zamrożony przedmiot nie będzie mógł wejść w interakcje z innymi obiektami, a ich fizyka będzie wyłączona. Użycie tej funkcji powinno być proste i intuicyjne, oparte na gestykulacji dłoni, która jest interpretowana przez kontrolery śledzące. Oparte o ten sposób wywołania będzie funkcja usuwania przedmiotów. By mieć możliwość usunięcia lub zamrożenia przedmiotu użytkownik będzie musiał wyciągnąć palec wskazujący i kliknąć odpowiedni przycisk. Przedmiot wycelowany przez palec będzie usunięty lub zamrożony. Proces tworzenia przedmiotu powinien mieć zabezpieczenie przed stworzeniem przedmiotu poza pomieszczeniem lub w innym obiekcie. Dlatego w momencie tworzenia obiekt może zostać stworzony jeśli nic nie będzie kolidować z nowo powstałym obiektem. Tworzenie przedmiotów będzie służyć do kreacji mebli, które są łatwe do przenoszenia oraz ich lokalizacja domyślnie nie jest stała. Przykładem takich mebli są krzesła, stoły, czy lampy. Z nimi użytkownik będzie głównie wchodzić w interakcje.

Zmiana wyglądu ścian jest ważna funkcją w projekcie, gdyż pozwala lepiej zwizualizować projektowane przez użytkownika pomieszczenie. Powinien on móc zmienić teksturę każdej ściany oraz nadać jej dowolny kolor. Wszystko z poziomu tabletu. Wystarczy go wycelować w pożądaną ścianę i ustawić konkretną teksturę i kolor. Tak jak liczba dostępnych tekstur będzie zależna od wymagań klienta to zmiana koloru będzie bazować na modyfikacji trzech podstawowych składników HSV.

Stałe obiekty są rodzajem obiektów, które są ściśle związane z daną ścianą. Nie można ich ręcznie przesunąć, a raczej trzeba jest ustawić w konkretnym miejscu. Przykładem takich obiektów są drzwi, okna, obrazy, czy nawet szafy. Użytkownik po postawieniu takiego obiektu, nie będzie wstanie go dalej przestawiać, czy modyfikować. Jedyną dostępną opcją będzie jego zniszczenie. W przeciwieństwie do zwykłych przedmiotów to stałe obiekty są tworzone na jednej konkretnej ścianie. Gdy użytkownik wybierze obiekt to do momentu zaakceptowania zmian, będzie mógł przesuwać obiekt w ramach jednej wybranej ściany. Biorąc pod uwagę różnice między różnymi obiektami to nie zawsze ruch stałego obiektu będzie dowolny. Przykładowo drzwi mogą się

przesuwać tylko w jednej osi, ponieważ musi stykać się z podłogą oraz ścianą, w przeciwieństwie do okna, która też musi mieć kontakt ze ścianą, ale nie z podłogą. Jednak najważniejszym i najbardziej szczególnym typem obiektu stałego jest dodatkowa ściana. Nie każdy pokój ma kształt prostopadłościanu, a często są mniejsze, czy większe nierówności lub pokój ma ścianę działkową. Postawienie takiej ściany daję użytkownikowi jeszcze większa wolność w tworzeniu pomieszczenia. Obiekt będzie się zachowywać jak zwykła ściana, czyli będzie można modyfikować jej teksturę, kolor oraz stawiać na niej kolejne stałe obiekty. Używając tabletu, użytkownik, będzie miał możliwość zwiększenia grubości, długości oraz obrotu nowo postawionej ściany. W przeciwieństwie do tworzenia samych przedmiotów to użytkownik powinien mieć możliwość stawiania stałych obiektów z dowolnie ustawiona precyzją. Do takich celów została przewidziana siatka oraz magnez. Pierwsza funkcja jest opcją, która po włączoniu nakłada na wszystkie ściany specjalną siatkę. Dzieli ona obiekty na mniejsze kwadraty o rozmiarze 20x20 jednostek, gdzie jedna jednostka odpowiada jednemu centymetrowi. Jest ona przydatnym narzędziem, by precyzyjnie ustawiać przedmioty w pomieszczeniu. Jednakże, by uzyskać pełnie precyzji przedmioty powinny się przyciągać do tej siatki. Rozwiązaniem jest magnez. Przy ustawianiu stałego przedmiotu jego lokalizacja będzie zaokrąglana o określoną przez użytkownika wartość. Ta funkcja zwiększy precyzje podczas projektowania pokoju.

Ostatnią funkcjonalnością jest widok z lotu ptaka. Użytkownik w każdym momencie będzie mógł się przełączyć między widokiem z pierwszej osoby na widok z lotu ptaka. W takiej pozycji użytkownik będzie mógł zobaczyć plan swojego pomieszczenia, ale nie będzie w stanie go modyfikować w znaczący sposób. Ta wizja jest czysto przeznaczona na podgląd zaprojektowanego pomieszczenia.

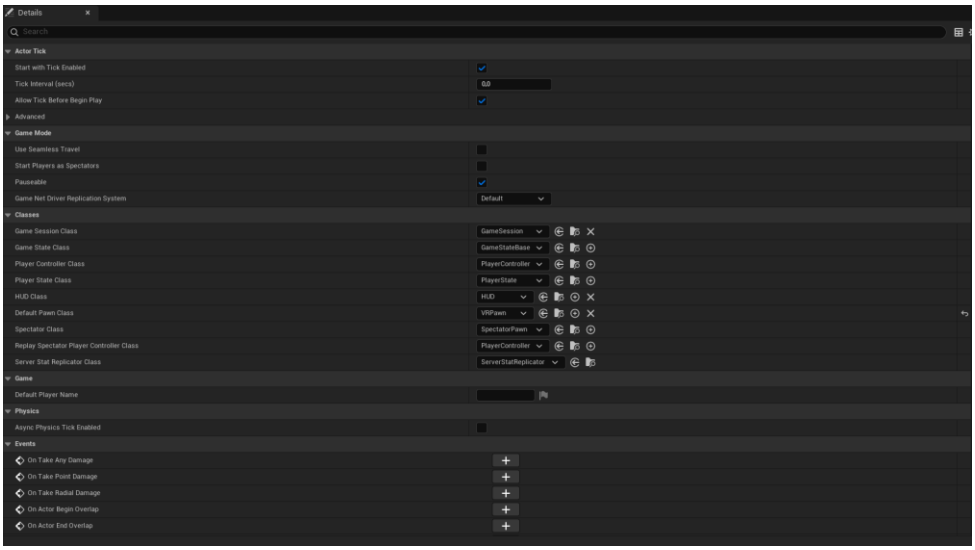
Cała ta funkcjonalność pozwoli na proste i intuicyjne zaprojektowanie dowolnego pomieszczenia. Wszystkie dostępne czynności użytkownik będzie mógł wykonać w naturalny sposób w wirtualnej rzeczywistości bez potrzeby dogłębnego zaznajomienia się z systemem. Projekt będzie natywnie działać na goglach Meta Quest 3, umożliwia to łatwiejszy dostęp do systemu w każdym miejscu.

3. Część Praktyczna

Implementacje systemu dekoracji wnętrz można podzielić na kilka etapów, które odwzorowują podstawowe funkcjonalności projektu. Zostanie przedstawiona ich implementacja oraz ze względu na wykorzystanie no code zostaną wyjaśnione wcześniej nieporuszone zagadnienia tego podejścia.

3.1 Implementacja Wirtualnej Rzeczywistości

Najpierw trzeba zacząć od tego w jaki sposób wirtualna rzeczywistość jest zaimplementowana do projektu stworzonego w Unreal Engine 5. Silnik oferuje wsparcie dla tej technologii oraz przykładowy projekt, gdzie najbardziej podstawowe mechaniki ruchu są już zaimplementowane. Dwoma głównymi elementami jest blueprint VRGameMode oraz aktor VRPawn. Pierwszy blueprint odpowiada za ustawienie domyślnych ustawień dla całego projektu.

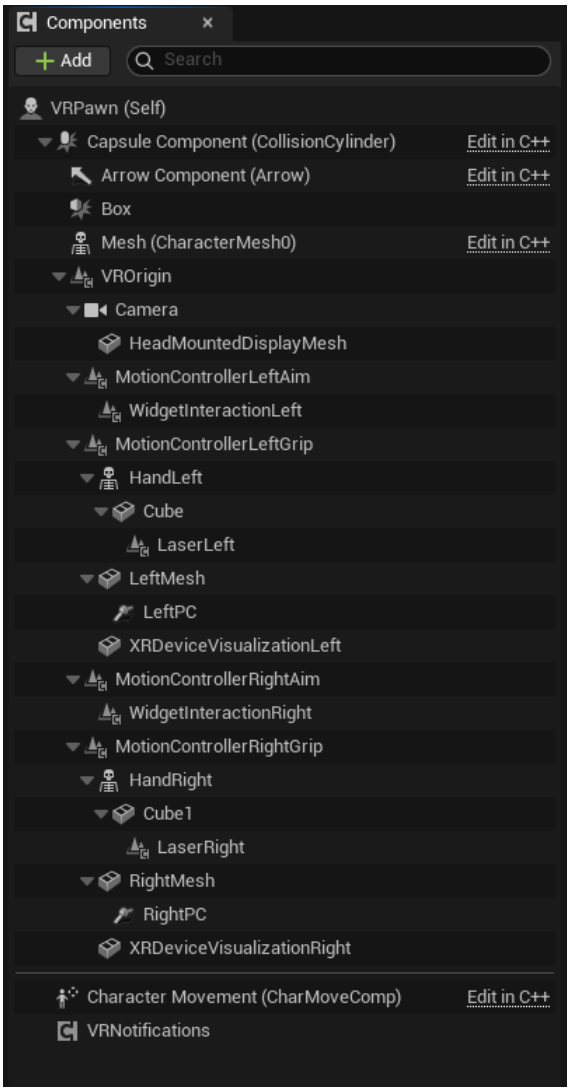


Rys. 3.1 Ustawienie VRGameMode. Źródło: opracowanie własne

Na rysunku tylko dwie opcje są znaczące dla tego projektu. Są to PlayerController oraz Default Pawn Class. Pierwsza opcja odnosi się do kontrolera użytkownika. Tu nie trzeba było nic zmieniać, ale jest to opcja, do której wielokrotnie będę się odwoływał. VRPawn to jest aktor, który reprezentuje użytkownika w przestrzeni trójwymiarowej. Z jego perspektywy użytkownik będzie wchodził w interakcje ze światem. VRGameMode poza tym nie posiada dodatkowego kodu, gdyż wszystkie ważne funkcje są operowane poprzez odpowiadającym im blueprinty.

VRPawn jest blueprintem typu Pawn, który jest kontrolowany przez użytkownika. Wszystkie operacje związane z interakcją z otoczeniem musza być

opracowane wewnątrz tego blueprinta. Pierwszą rzeczą jaką trzeba omówić to drzewo komponentów VRPawn.



Rys. 3.2 Drzewo komponentów VRPawn. Źródło: opracowanie własne

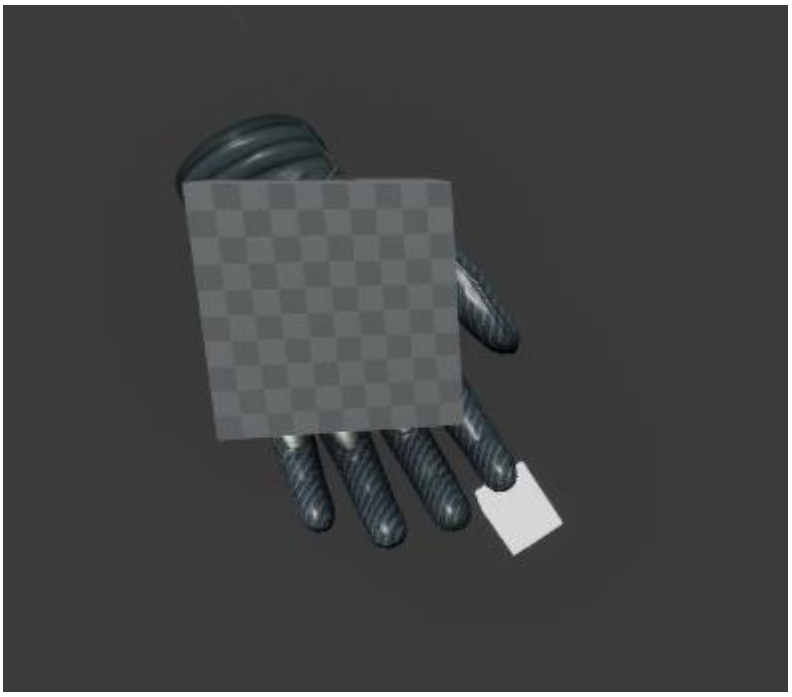
VRPawn dziedziczy po wbudowanej klasie Pawn, więc kilka komponentów jest domyślnie ustawione w drzewie. Głównym komponentem jest Capsule Component. Jest on odpowiedzialny za kolizje z innymi obiektami. Można powiedzieć, że określa rozmiar użytkownika. Wszystkie inne komponenty są od niego zależne w relacji rodzic-dziecko. Jeden komponent może mieć jednego rodzica oraz wiele dzieci. Arrow Component jest dzieckiem Capsule Component. Odpowiada on za orientacje pionka w przestrzeni 3D. Pierwszym niedziedzicznym komponentem w drzewie jest Box. Jest to kolejny komponent odpowiadający za kolizje. Różnica między Capsule Component, a Box jest ich zastosowanie. Capsule Component służy do wykrywania kolizji ścian, by zapobiec niepożądanym

przenikaniu przez obiekty. Box z drugiej strony odpowiada za wykrywanie kolizji stworzonych przez użytkownika przedmiotów. Zdecydowałem się na takie rozwiązanie głównie ze względu działania fizyki w Unreal Engine 5. Przez często nieregularną geometrię obiektów zdarzały się sytuacje, gdzie lekkie przesunięcie ciałem obiektu skutkowało wystrzeleniem go z ogromną siłą. Zastosowanie dodatkowego wykrycia kolizji tylko dla tych obiektów usunęło ten problem. Mesh jest komponentem dziedzicznym po Pawn i nie ma on funkcji w tym blueprincie, ale też nie jest możliwe jego usunięcie ze względu na wcześniej wspomnianą dziedziczność. VROrigin to komponent, który jest rodzicem dla wszystkich komponentów związanych z obsługą wirtualnej rzeczywistości. Camera odpowiada za widok użytkownika. Są one jego cyfrowymi oczami i przez nie użytkownik będzie obserwował środowisko 3D dodatkowo jest ona zwizualizowana przez komponent static mesh, który może reprezentować dowolny model 3D. W tym wypadku jest to model 3D gogli VR. Użytkownik tego nie widzi, ale sam ten fakt okazał się pomocny przy tworzeniu projektu. Za wykrywanie i przenoszenie ruchu rąk gracza w wirtualne środowisko odpowiadają MotionControllerComponent. VRPawn posiada aż cztery takie komponenty po dwa na każdą rękę. Jeden z tych komponentów dla przykładu MotionControllerLeftAim odpowiada za interakcje lewej ręki z blueprintem typu widget, o którym więcej będzie przy implementacji generacji pomieszczeń. Ważniejszym komponentem dla projektu jest ten odpowiadający za chwytanie przedmiotów. Dla przykładu MotionControllerLeftGrip odpowiada za wizualizacje lewej ręki, która będzie chwytać obiekty w przestrzeni 3D. To czy prawa, czy lewa ręka będzie przez ten komponent wykrywana odpowiada opcja Motion Source.

Rys. 3.3 Opcja Motion Source dla MotionControllerLeftAim. Źródło: pracowanie własne

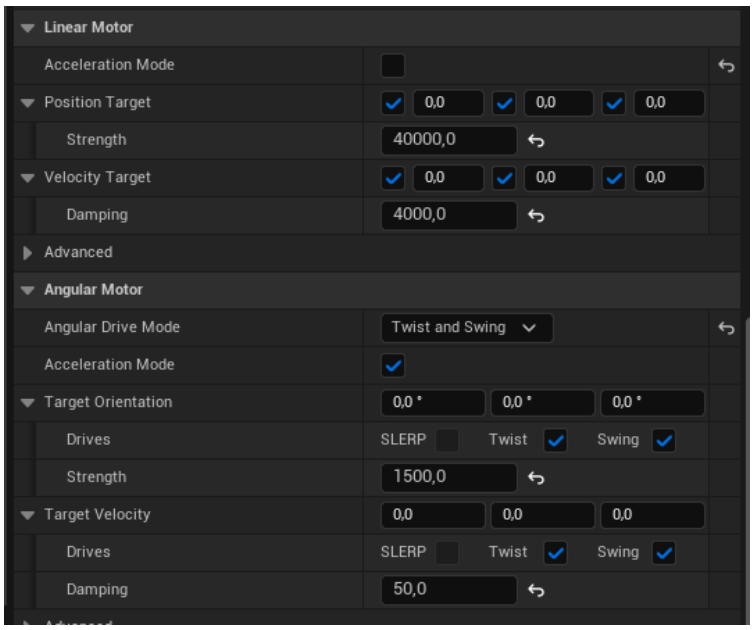
Największym problemem przy implementacji wizualizacji rąk było wykrywanie kolizji rąk. Domyślne rozwiązanie Unreal Engine 5 oferuje prostą wizualizację rąk,

które przenikają przez obiekty. By umożliwić użytkownikowi interakcje z innymi obiektami musiałem inaczej zaprojektować całą obsługę rąk. Zamiast zmieniać pozycje i obrót samego modelu dłoni na podstawie ruchu rąk użytkownika to całe wejście jest przekazywane do kostki, która jest punktem odniesienia dla modelu dłoni.



Rys. 3.4 Lewa Ręka Użytkownika. Źródło: opracowanie własne

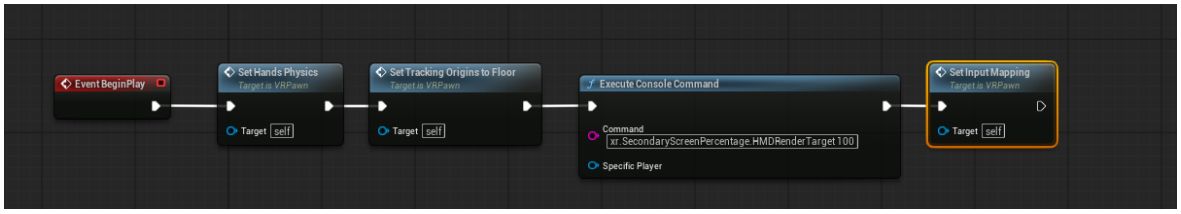
Kostka precyzyjnie wizualizuje w przestrzeni 3D ruch użytkownika. Model 3D dłoni zawsze będzie próbować podążać za kostką. Za tą funkcjonalność odpowiada Physics Constraint Component, który łączy oba obiekty. Dzięki temu rozwiązaniu model 3D wizualizuje ruch rąk użytkownika przy jednoczesnym umożliwieniu interakcji fizycznej z innymi obiektami. Sama kostka nie jest widoczna dla użytkownika i jedyne co obserwuje to płynny ruch dłoni. Jako że model dłoni podąża za punktem odniesienia to pojawiają się mniejsze nie dokładności szczególnie przy poruszaniu się. By zniwelować je trzeba było dostosować parametry Physics Constraint Component.



Rys. 3.5 Parametry Physics Constraint Component. Źródło: opracowanie własne

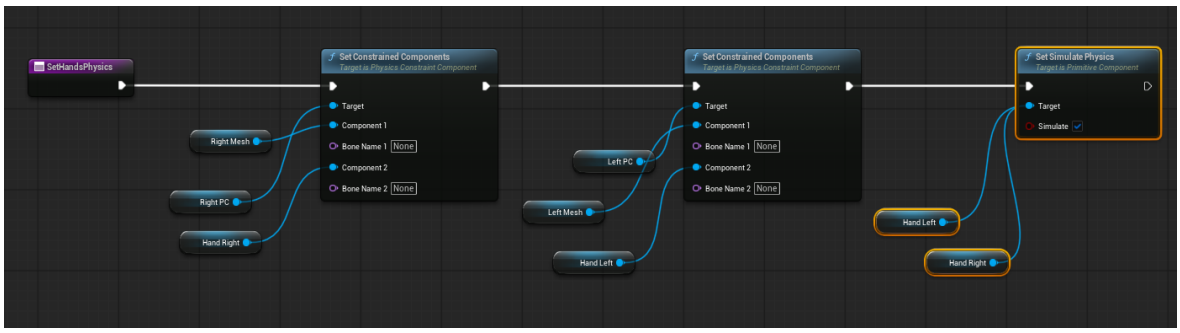
Ustawienie dużych wartości powoduje, że model 3D dłoni natychmiastowo będzie się przyciągać do pozycji kostki. Nadal jako że to jest obiekt fizyczny pojawiają się niedokładności przy ruchu, ale są one minimalne. Ostatnim elementem wizualizującym ręce użytkownika jest dodatkowa biała kostka służąca za punkt odniesienia dla palca wskazującego. Na podstawie tej kostki są rysowane wskaźniki ułatwiające celowanie. Obie dłonie działają w ten sposób.

Mając drzewo komponentów omówione można przejść do kodu, który realizuje implementacje wirtualnej rzeczywistości.



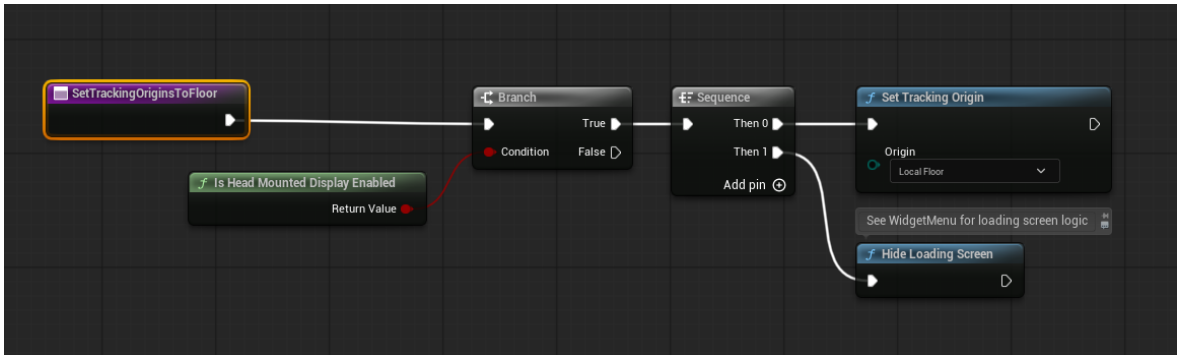
Rys. 3.6 Konstruktor VRPawn. Źródło: opracowanie własne

Gdy obiekt VRPawn zostanie stworzony te cztery bloki zostaną wykonane jako pierwsze. Blok Set Hands Physics odpowiada za połączenie punktu odniesienia rąk z modelem dłoni za pomocą Physics Constraint Component.



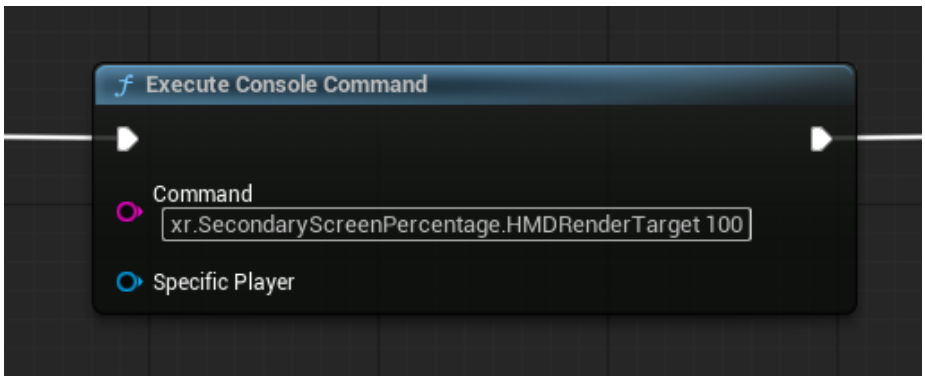
Rys. 3.7 Funkcja Set Hands Physics. Źródło: opracowanie własne

Funkcja ustawia połączenie dla obu dłoni używając wbudowanego bloku Set Constrained Components. Przyjmuje ona trzy parametry: Model Dłoni, punkt odniesienia oraz referencje do Physics Constraint Component. Po ustawieniu więzów jest aktywowana symulacja fizyki dla modelów dłoni.



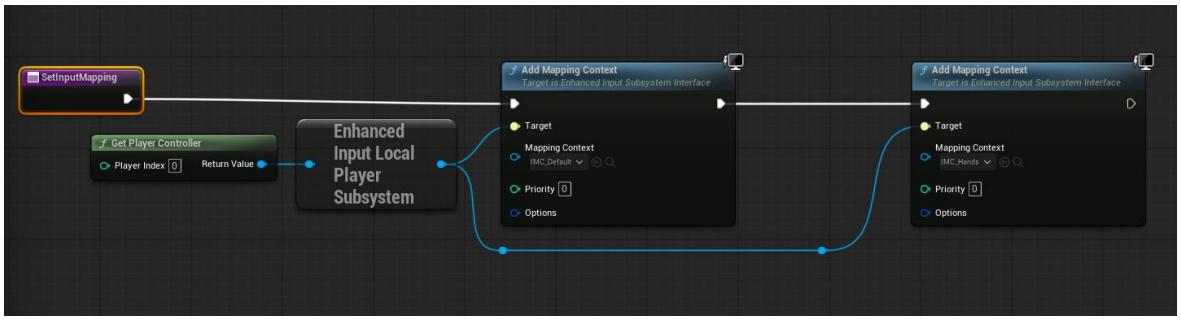
Rys. 3.8 Funkcja Set Tracking Origins To Floor. Źródło: opracowanie własne

Jest to najważniejsza funkcja VRPawn ponieważ ustawia ona jak będzie zachowywać się kamera. Aplikacja zakłada, że użytkownik będzie z niej korzystał na stojąco. Oznacza to, że wysokość kamery jest zależna od wysokości użytkownika. W tym celu kamera w VRPawn jest ustawiona na poziomie 0 dla osi Z. Daje to efekt, że kamera leży na podłodze. Funkcja po wykryciu gogli VR ustawia źródło śledzenia na lokalną podłogę. Powoduje to, że kamera automatycznie dostosowuje się do wysokości użytkownika. Po tym ukrywany jest ekran ładowania dla poprawnego działania programu.



Rys. 3.9 Blok wykonujący komendę w konsoli. Źródło: opracowanie własne

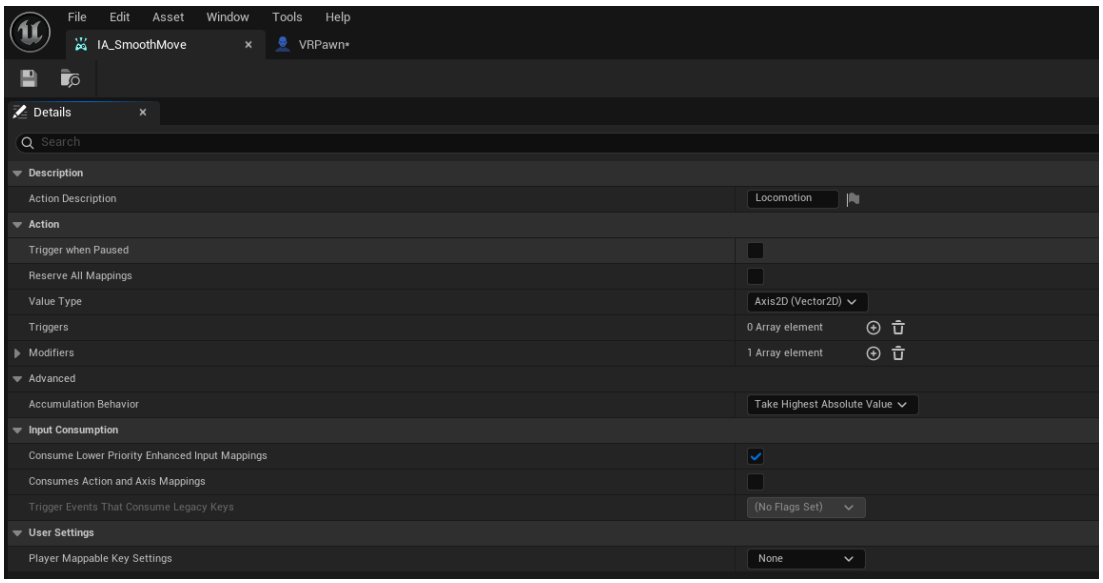
Ten blok wykonuje podaną komendę w konsoli. W tym wypadku ustawia to rozdzielczość poglądu aplikacji na monitorze.



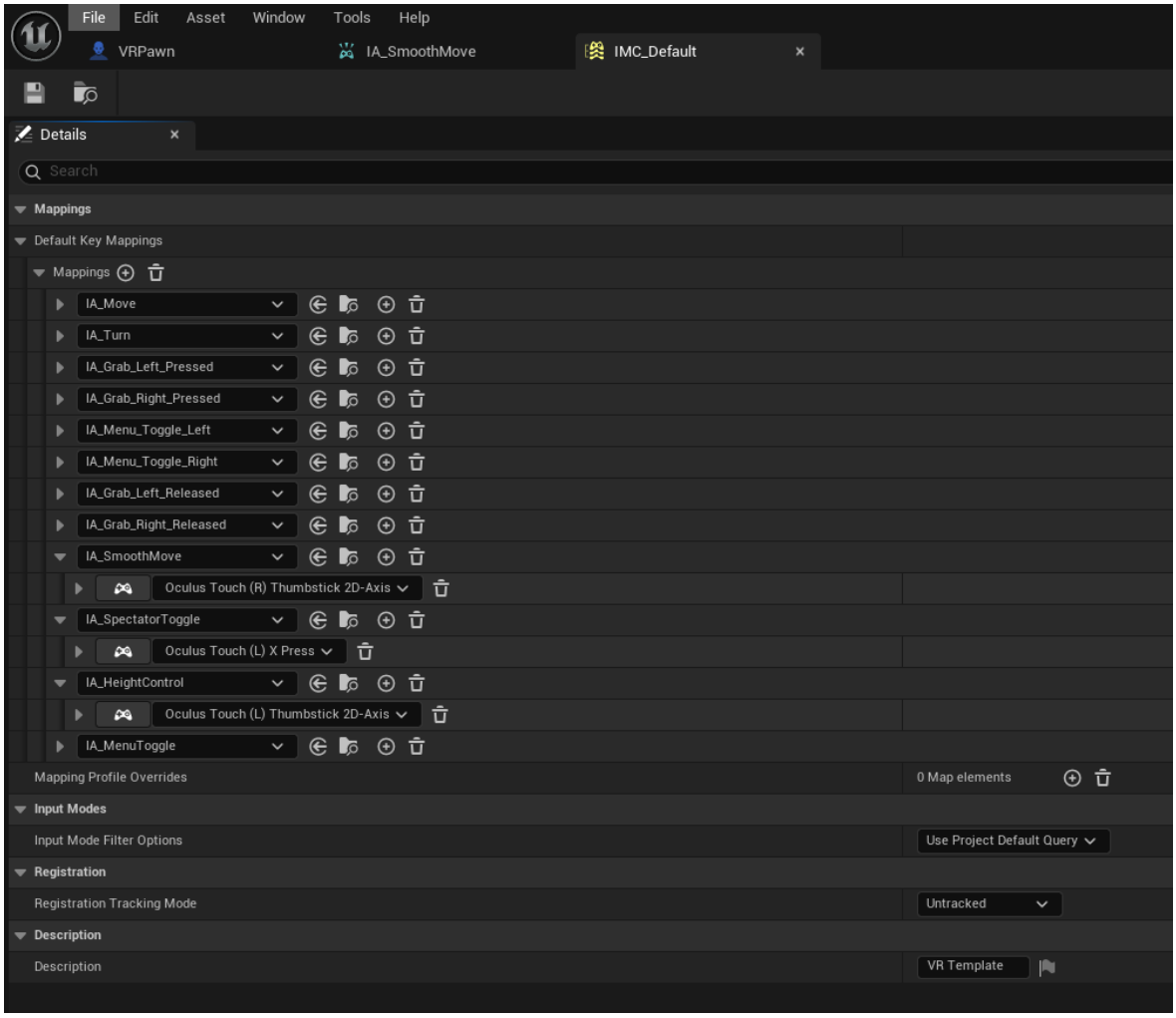
Rys. 3.10 Funkcja Set Input Mapping. Źródło: opracowanie własne

Ta funkcja odpowiada za mapowanie ustawień przycisków kontrolera VR do Player Controller. Są ustawiane dwa mapowania za pomocą bloków Add Mapping Context, które przyjmują jako parametry podsystem wejść użytkownika oraz kontekst mapowania. Jedno jest mapowane dla domyślnych ustawień przycisków kontrolera VR, z których będzie korzystał użytkownik. Natomiast drugie posłuży obsłudze animacji modelu dłoni użytkownika.

Do wykrycia faktu, że użytkownik wciska przyciski na kontrolerze wykorzystałem Enhanced Input Action, czyli rozwiązanie Unreal Engine 5 do przechwytywanie wejść kontrolera. By skorzystać z tego rozwiązania trzeba stworzyć akcje wejść oraz przypisać je do konkretnych przycisków kontrolera.

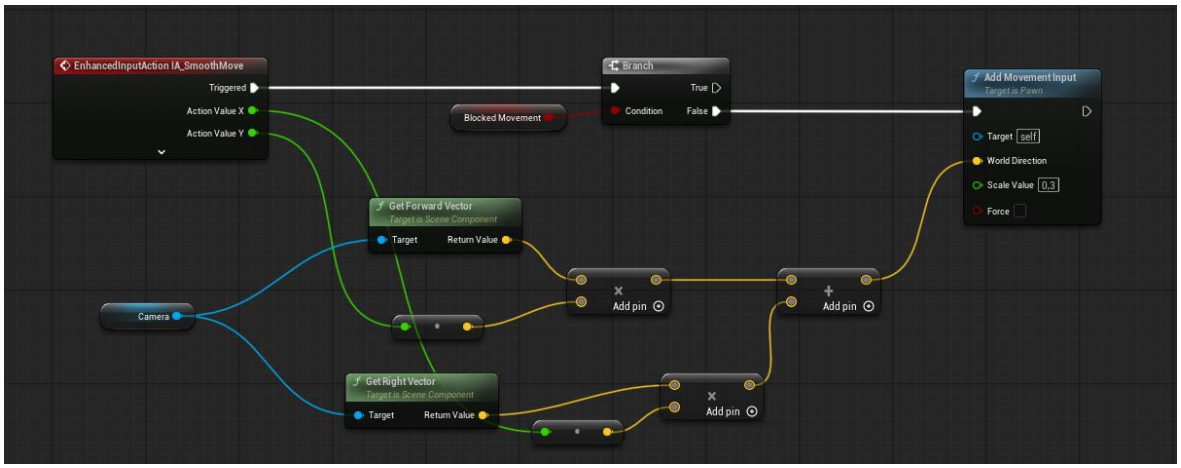


Rys. 3.11 Akcja wejścia dla płynnego poruszania. Źródło: opracowanie własne



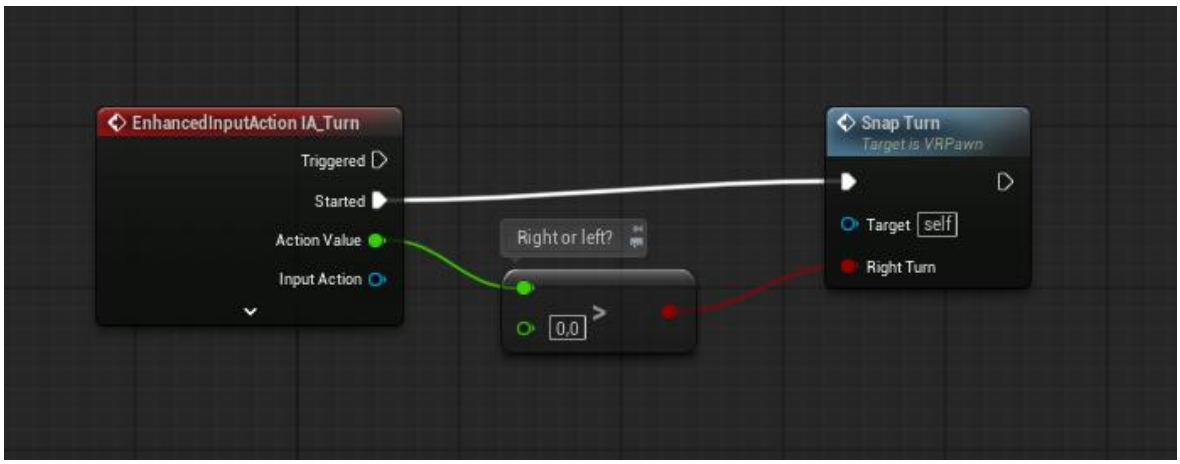
Rys. 3.12 Mapowanie ustawień kontrolera VR. Źródło: opracowanie własne

Każda akcja zwraca jakąś wartość zależnie od wyboru jej typu. Podczas mapowania wybiera się przycisk który będzie obsługiwał daną akcję.



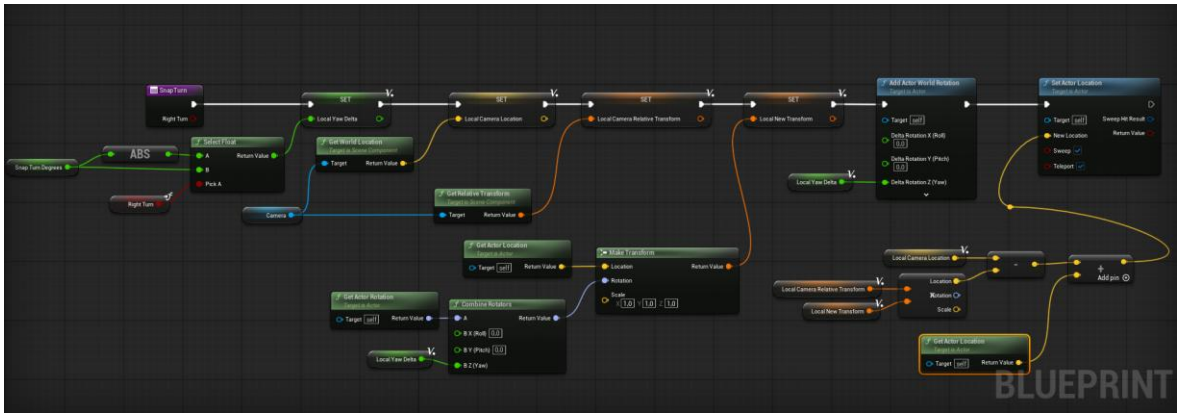
Rys. 3.13 Implementacja płynnego poruszania. Źródło: opracowanie własne

Po załączeniu wejść kontrolera można stworzyć blok typu event, który aktywuje się gdy użytkownik poruszy gałkę analogową prawego kontrolera. Blok zwraca dwie wartości, które odpowiadają kierunkowi w jakim gałka została przemieszczona. Gdy blok zostanie wywołany zostanie sprawdzony warunek, czy Blocked Movement jest prawdą. Zmienna Blocked Movement określa czy poruszanie użytkownikowi ma być zablokowane czy nie. Jeśli nie to jest wywoływany jest Blok Add Movement Input, który przyjmuje dwa argumenty. World Direction oraz Scale Value. World Direction określa kierunek w jakim ma być VRPawn przesunięty. World Direction jest sumą dwóch wektorów, które określają kolejno przesuniecie w osi X i Y. Pobierany jest wektor do przodu kamery VRPawn i jest to mnożone przez wartość X bloku typu event. To samo tyczy się dla osi Y, tylko zamiast wektora do przodu jest pobierany wektor w prawo. Scale Value określa prędkość z jaką jest VRPawn przesuwany. Jest ustawiona niską wartość, by użytkownik nie był wystawiony na szybkie przemieszczanie się, które mogłoby sprowokować chorobę symulatorową.



Rys. 3.14 Implementacja obrotu skokowego. Źródło: opracowanie własne

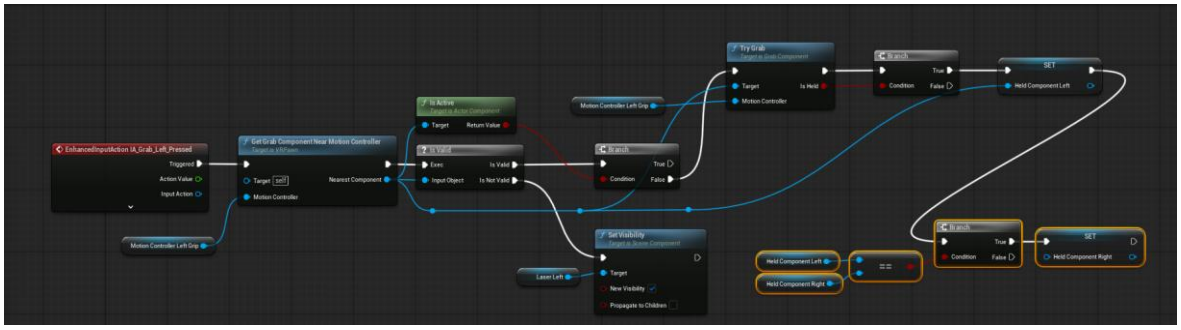
Użytkownik z założenia może obrócić się kierując głowę w wybranym kierunku, ale biorąc pod uwagę fakt możliwości braku swobodnego obrotu postanowiłem zaimplementować obracanie skokowe. Wychylając gałkę analogową lewego kontrolera użytkownik może sztucznie się obrócić o określoną liczbę stopni. Tak jak w przypadku płynnego poruszania jest wywoływany blok typu event, który reaguje, gdy gałka zostanie wychylona. Jednakże jest zwracana tylko jedna wartość, która przyjmuje wartości dodatnie, gdy gałka jest wychylona prawo oraz ujemne w przeciwnym wypadku. Gdy użytkownik wychyli gałkę zostanie wywołana funkcja Snap Turn.



Rys. 3.15 Funkcja Snap Turn. Źródła: opracowanie własne

Funkcja Snap Turn nie tylko musi obrócić całego VRPawn to jednocześnie musi przestawić całego VRPawn, by odpowiadał pozycji kamery. Podczas obrotu kamery jego pozycja się lekko zmienia. By uniknąć znacznego oddalenia kamery od całego VRPawn funkcja musi dostosować pozycję kamery i VRPawn. W

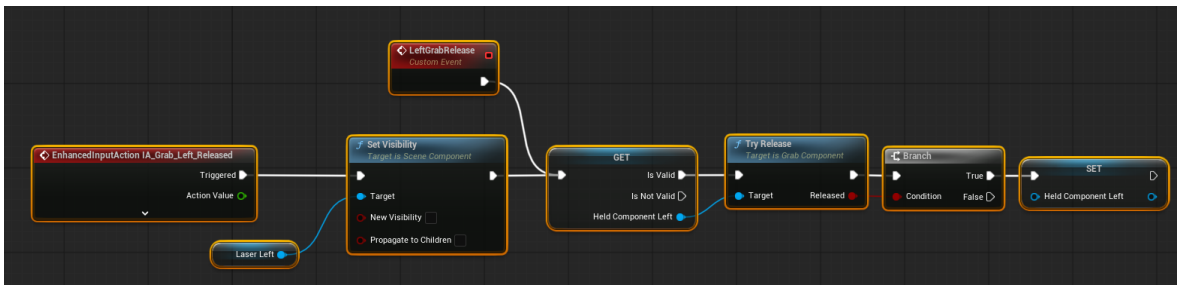
pierwszej kolejności jest określany kierunek obrotu. Zmienna Local Yaw Delta przechowuje kąt obrotu, który jest decydowany na podstawie parametru Right Turn oraz Snap Turn Degrees. Snap Turn Degress jest zmienną typu float i domyślnie przechowuje wartość -45.0 stopni. Jeśli Right Turn jest prawdą to do Local Yaw Delta jest ustawiana na wartość bezwzględną Snap Turn Degress, w przeciwnym wypadku jest ustawiana wartość bez zmian. Kolejne bloki odpowiedzialne są za ustawienie zmiennych pomocniczych. Local Camera Location jest zmienną typu vector, która przechowuje lokalizacje kamery w środowisku 3D. Local Camera Relative Transform jest zmienną typu transform, która składa się z trzech wektorów odpowiadającym kolejno za lokalizacje, rotację oraz skale. Jest ona ustawiana na relatywną transformacje kamery. Local New Transform też jest zmienną typu transform, ale przechwoje transformacje całego VRPawn z dodanym przesunięciem obrotu o Local Yaw Delta. Po ustawieniu wszystkich zmiennych pomocniczych jest wywoływany blok Add Actor World Rotation, który dodaje do aktualnej rotacji VRPawn Local Yaw Delta, następnie jest wywoływany blok Set Actor Location ustawiającym lokalizacje VRPawn. Nowa lokalizacja jest obliczana na podstawie sumy aktualnej lokalizacji VRPawn i różnicy Local Camera Location i ilorazu Local Camera Ralative Transform i Local New Transform. W ten sposób użytkownik za pomocą lewej gałki kontrolera może skokowo obracać się w przestrzeni 3D.



Rys. 3.16 Implementacja łapania przedmiotów. Źródło: opracowanie własne

Kluczowym elementem implementacji wirtualnej rzeczywistości jest możliwość podnoszenia obiektów w przestrzeni 3D. Unreal Engine 5 ma domyślnie zaimplementowaną całą funkcjonalność do tego zadania. Jedyne co trzeba obsłużyć to zdarzenie kliknięcia przycisku odpowiedzialnego za podnoszenie. Po wykryciu tego zdarzenia jest wywoływana funkcja Get Grab Component Near

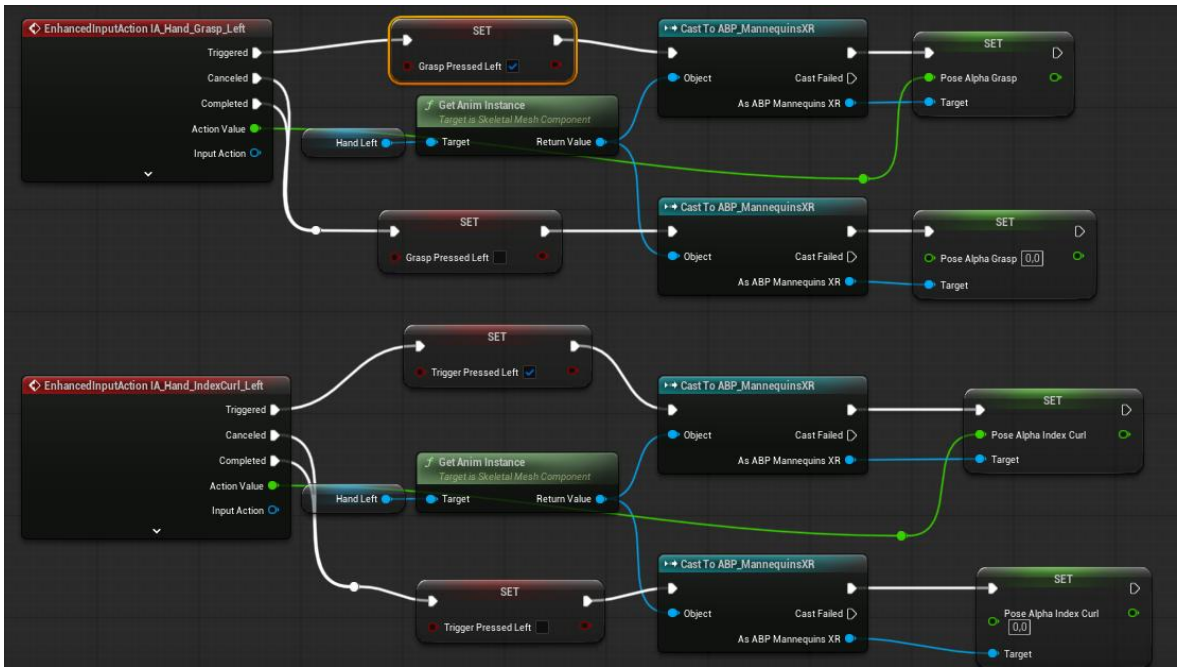
Motion Controler. Jest to funkcja wbudowana w Unreal Engine 5 i zwraca referencje do Grab Component najbliższego obiektu. Grab Component jest specjalnym komponentem, który nadaje dowolnemu obiektowi możliwość bycia podniesionym. Po wywołaniu funkcji jest sprawdzane, czy wracana wartość jest prawidłowa, czyli czy obiekt posiada Grab Component. Jeśli nie jest to wyświetlany jest laser z palca wskazującego. Natomiast jeśli obiekt w okolicy dłoni posiada Grab Component to jest sprawdzany warunek, czy jest on aktywny. I dalsza funkcjonalność jest wywoływana kiedy komponent jest nieaktywny. To jest nie jasny błąd silnika, który ustawia aktywność Grab Component domyślnie na false. To czy komponent jest aktywny czy nie, nie ma znaczenia dla jego działania, ale w ten sposób mogę określić, czy przedmiot z tym komponentem ma być podnoszony lub nie. Jeśli nie chce, by można było go podnosić to ustawiam aktywność jego komponentu na true. Gdy użytkownik spróbuje podnieść przedmiot, który ma nieaktywny Grab Component to jest wywoływana funkcja Try Grab. Jest to wbudowana funkcja w Grab Component, która próbuje przymocować obiekt z komponentem do ręki użytkownika. Jeśli operacja się powiodła to referencje do obiektu jest przypisywana do zmiennej Held Component X, gdzie X oznacza lewą lub prawą rękę. Jeśli się okaże, że użytkownik próbuje drugą ręką podnieść trzymany obiekt, czyli Held Component Left i Right mają taką samą wartość to zmienna dotycząca ręki, która pierwotnie trzymała obiekt jest czyszczona. Daje to efekt, że obiekt przeskakuje z jednej ręki do drugiej.



Rys. 3.17 Implementacja puszczania przedmiotu. Źródło: opracowanie własne

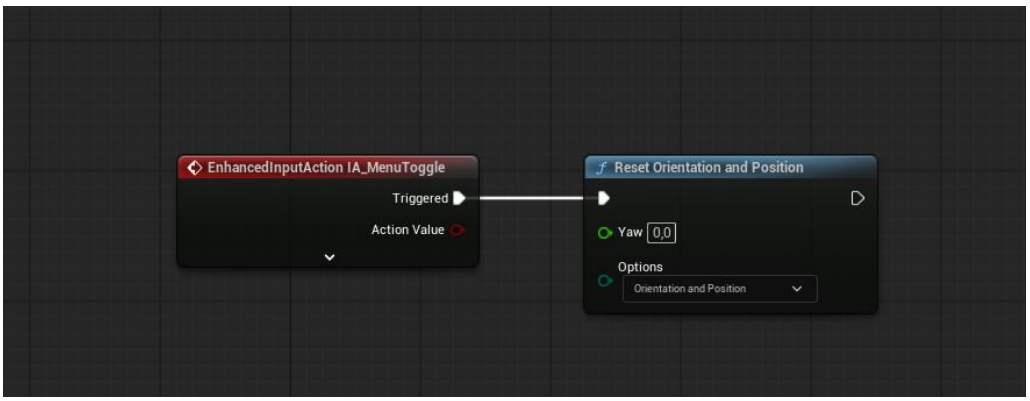
W wypadku, gdy użytkownik puści przycisk na kontrolerze odpowiedzialny za trzymanie obiektu to powinien on zostać puszczony z ręki w środowisku 3D. Pierwszą jednak rzeczą jaką ta funkcja robi to ustawia widoczność lasera odpowiadającej ręki. Następnie jest sprawdzane, czy zmienna Held Component X zawiera referencje do komponentu. Jeśli zawiera to jest wywoływana wbudowana

funkcja Try Release, która wypuszcza z ręki przedmiot. Jeśli operacja się powiodła to jest czyszczona zmienna przechowująca referencje do Grab Component obiektu. Ta funkcjonalność jest wywoływana poprzez puszczenie przycisku, ale i również przez zwykłe wywołanie. Od tego jest blok Left/Right Grab Release. Jest on punktem końcowym, do którego inne funkcję mogą się odwoływać, by zmusić użytkownika do puszczenia trzymanego przedmiotu. Ta cała funkcjonalność jest zaaplikowana dla obu rąk.



Rys. 3.18 Przykładowy kod obsługujący gestykulacje. Źródło: opracowanie własne

Gestykulacja rąk jest obsługiwana przez zbiór bloków typu event wywoływane gdy wykryty zostanie dotyk konkretnych przycisków kontrolera VR. Wtedy zostanie zmienione ustawienie palców na modelu 3D. To jaki palec jest kontrolowany przez jaki przycisk ma odwzorowywać prawdziwą gestykulacje użytkownika podczas trzymania kontrolera VR. To jest funkcjonalność domyślnie zaimplementowana przez Unreal Engine 5.

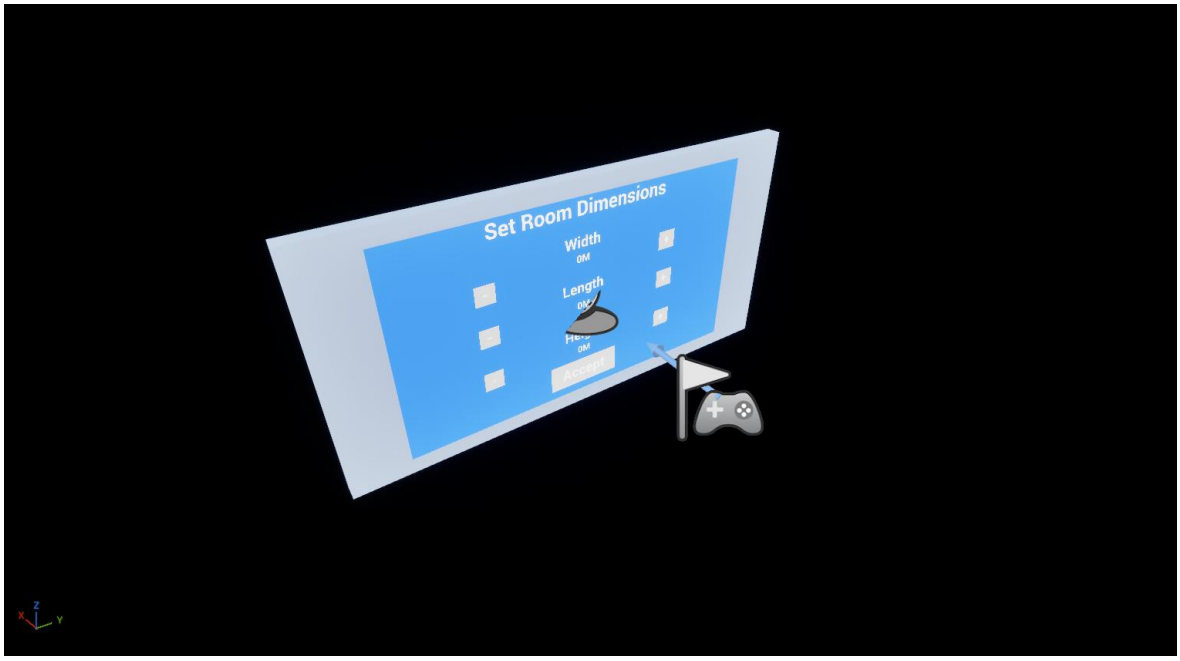


Rys. 3.19 Implementacje resetu orientacji. Źródło: opracowanie własne

Ta prosta opcja uruchamiana po naciśnięciu przycisku A na prawym kontrolerze spowoduje reset orientacji i pozycji użytkownika. Blok Reset Orientation and Position jest zbudowanym blokiem w Unreal Engine 5. Rozwiązuje to problem przesunięcia użytkownika względem pozycji i obrotu VRPawn. Gdy użytkownik przemieści się w realnym świecie to kamera w programie też się przemieści. Jednak VRPawn zostanie tam gdzie był wcześniej. To samo tyczy się obrotu. Może to doprowadzić do mylnego interpretowania przez program pozycji użytkownika. Opcja dostosowuje pozycje i obrót użytkownika do pozycji i obrotu VRPawn.

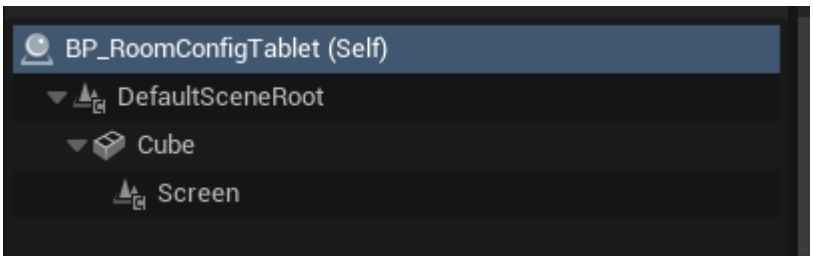
3.2 Generowanie Pomieszczenia

Pierwszą czynnością jaką wykonuje użytkownik po uruchomieniu programu jest generacja pomieszczenia. Użytkownik podaje rozmiary pokoju, a po akceptacji trafia do nowo stworzonego pomieszczenia.



Rys. 3.20 Pomieszczenie do generacji pokoi. Źródło: opracowanie własne

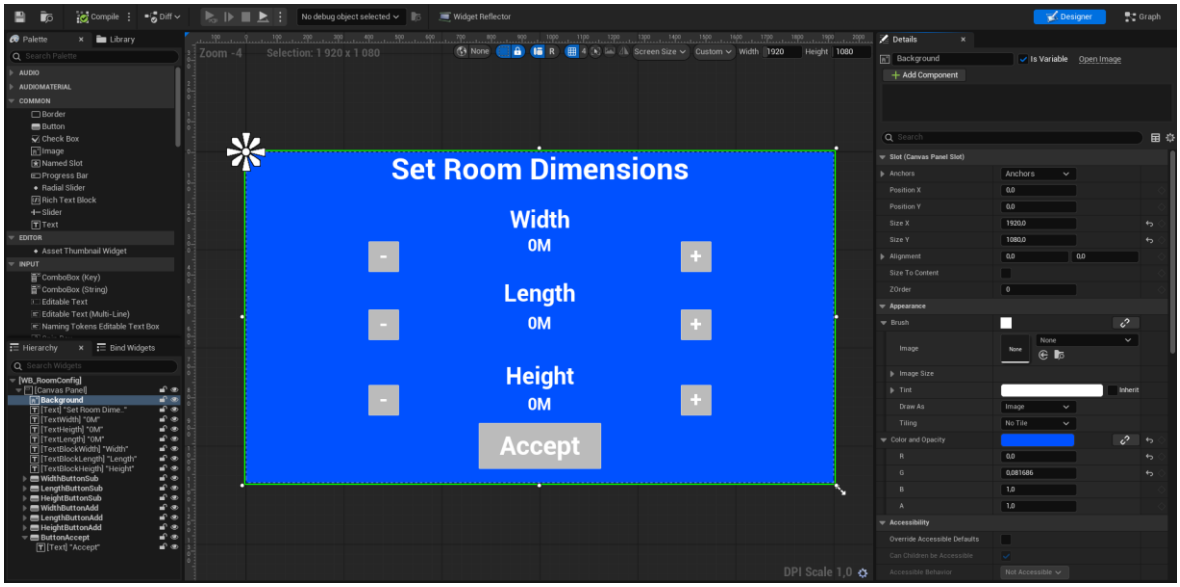
Ustawianie wymiarów pomieszczenia odbywa się za pośrednictwem specjalnego obiektu BP_RoomConfigTablet. Jest to blueprint typu actor, z którym użytkownik może wejść w interakcje.



Rys. 3.21 Drzewo komponentów BP_RoomConfigTablet. Źródło: opracowanie własne

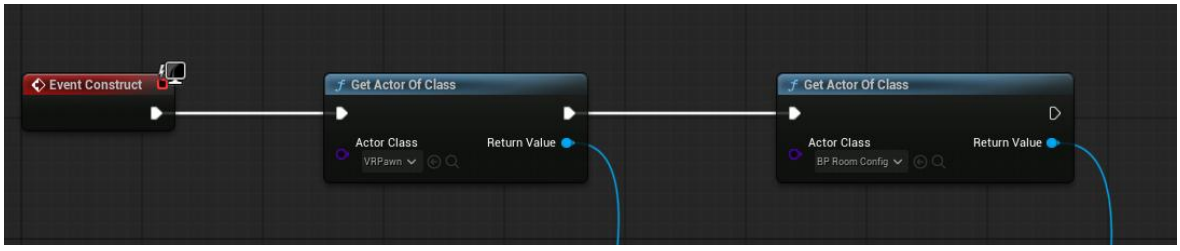
Komponent Cube jest obiektem na którym jest wyświetlany interfejs. Stanowi on też blokadę dla dłoni, by nie przenikały przez ekran. Ma on w sobie jeden komponent dziecko, który jest odpowiedzialny za wyświetlenie interfejsu w przestrzeni trójwymiarowej. Interfejs jest zaprojektowany przy pomocy innego typu

blueprint. Do tworzenia interfejsów użytkownika 2D wykorzystuje się widgety. Różnią się od zwykłego typu blueprint tym, że można w nich zaprojektować cały interfejs 2D.



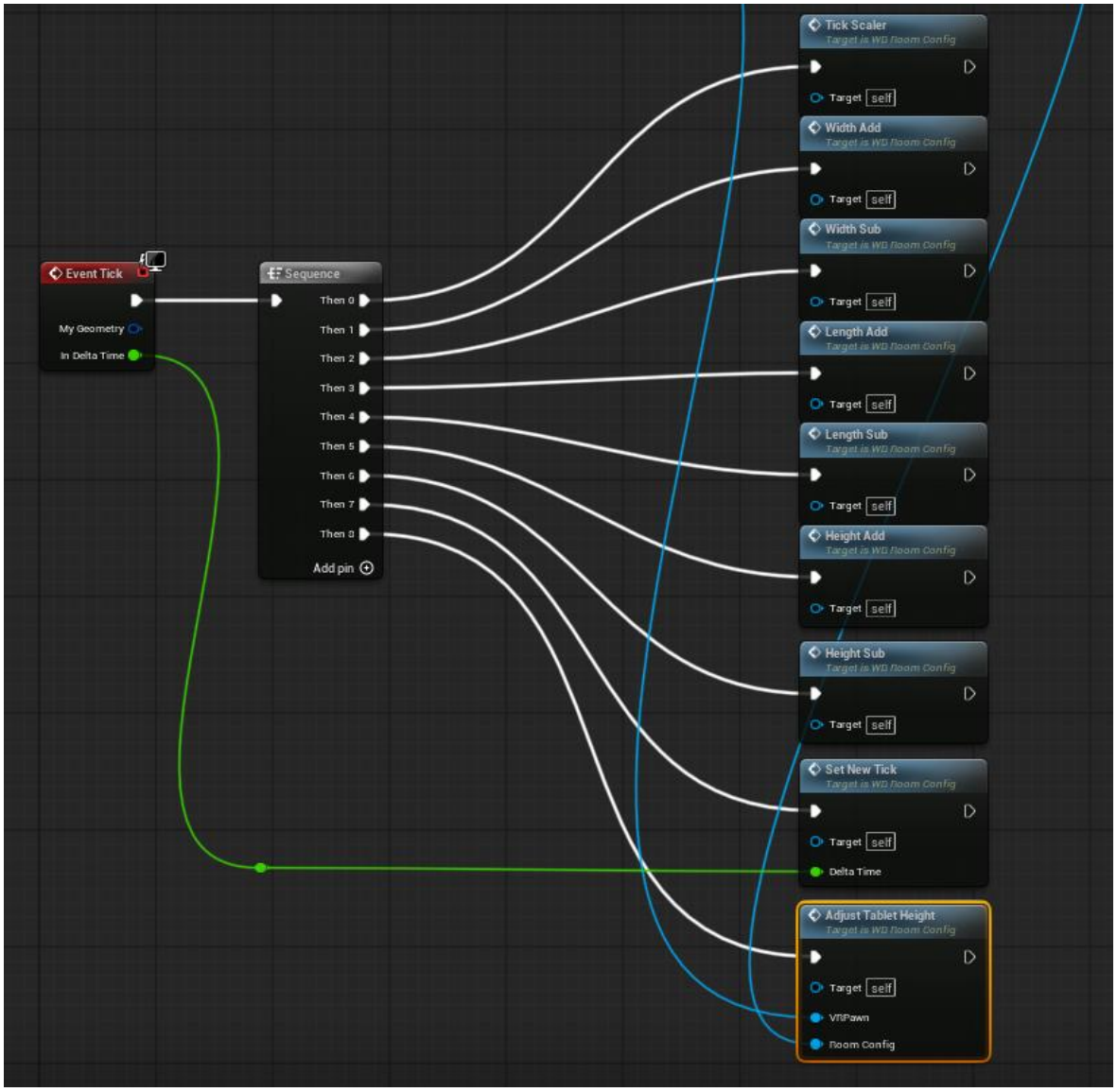
Rys. 3.22 Projektowanie Widget. Źródło: opracowanie własne

Interakcja użytkownika z widgetem obywa się za pośrednictwem komponentu VRPawn MotionControlerXAim, gdzie X odpowiada ręce użytkownika. Przez palec wskazujący ręki jest rysowany promień, który umożliwia interakcje z widgetami. Im dłuższy promień tym z większej odległości użytkownik może wejść w interakcje. Na potrzeby zasymulowania interakcji dotykowej, promień jest ustawiony na minimalną wartość. W ten sposób użytkownik musi dotknąć przycisku, by wejść z nim w interakcje. Widget odpowiedzialny za generacje pomieszczenia ma trzy wartości, które są modyfikowalne przez przyciski po obu stronach oraz przycisk do akceptacji wymiarów. Cała logika tworzenia jest obsługiwana przez ten widget. Projektowanie logiki widgetu jest identyczne jak w zwykłym blueprincie.



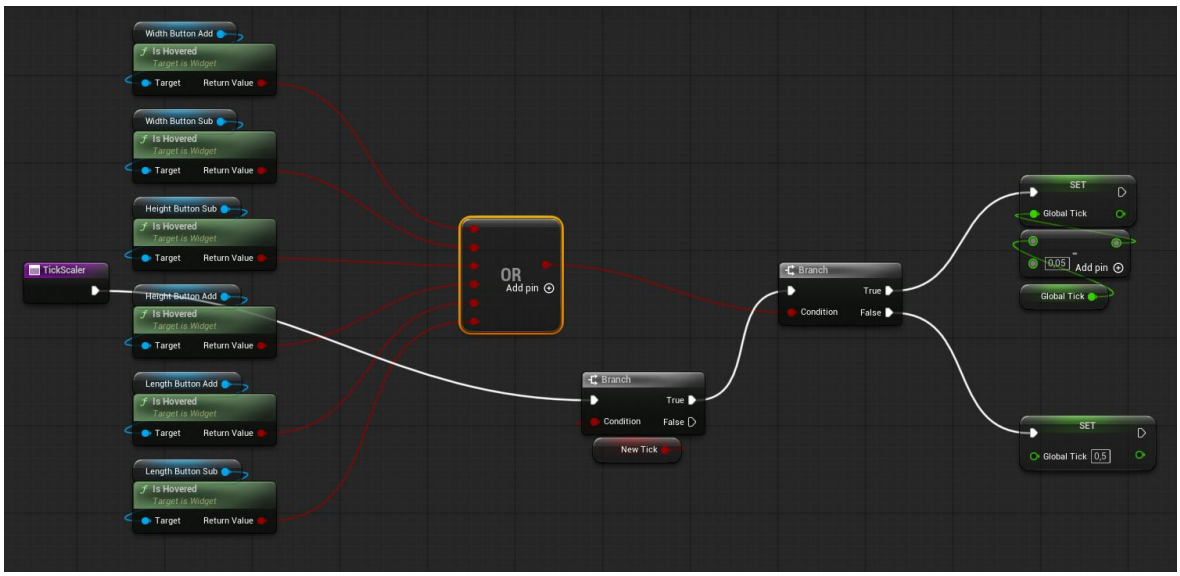
Rys. 3.23 Konstruktor Widgetu. Źródło: opracowanie własne

Gdy widget jest tworzony to wyszukuje dwa obiekty na mapie. Get Actor Of Class wyszukuje te obiekty i zwraca do nich referencje. Dzięki temu z poziomu blueprintu można mieć dostęp do zmiennych i funkcji tych klas.



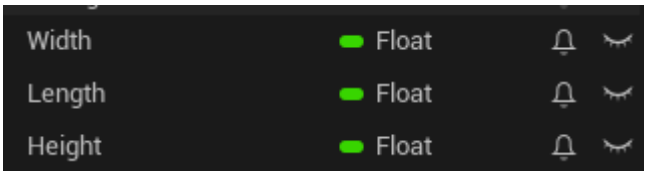
Rys. 3.24 Event Tick. Źródło: opracowanie własne

Event Tick jest wywoływany co określony przedział czasowy. Gdy zostanie on wywołany zostaną wykonane kolejno funkcje odpowiedzialne za ustawienie zmiennych i obsługę przycisków. Ważnym aspektem zmiany wartości zmiennych jest precyzja. W założeniu im dłużej użytkownik będzie trzymał przycisk tym szybciej będzie się zwiększać wartość zmiennej. By to zaimplementować musiałem skalować domyślny przedział czasowy tick, by wykonywać operacje w coraz to szybszym czasie.



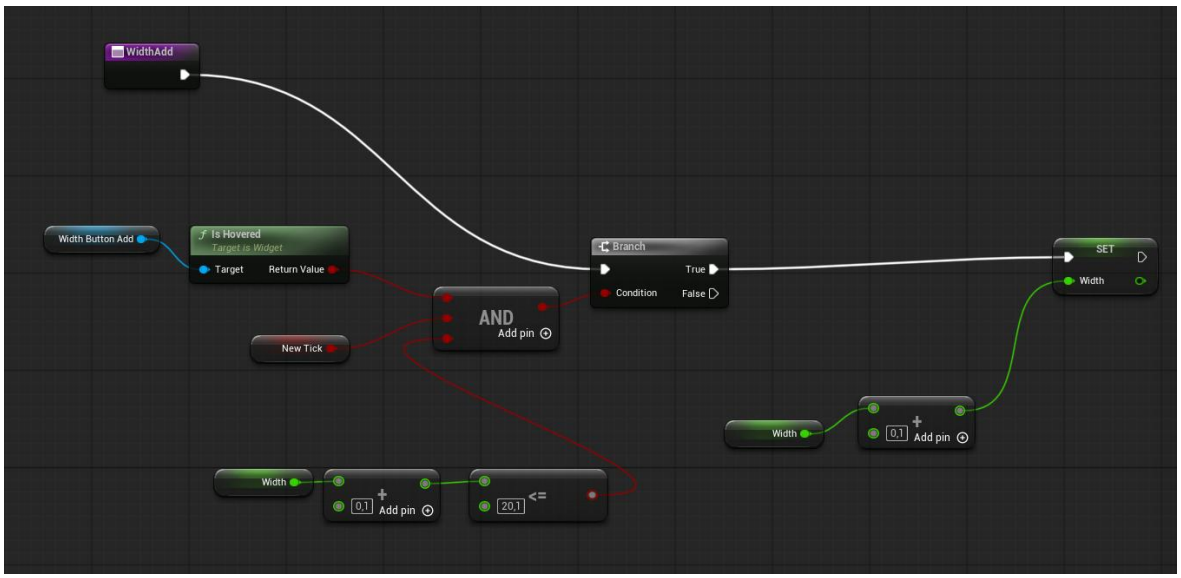
Rys. 3.25 Funkcja Tick Scaler. Źródło: opracowanie własne

Funkcja Tick Scaler odpowiada za skalowanie przedziału czasowego w zależności od długości trzymania przycisku. Pierwszą operacją wykonywaną przez funkcje jest sprawdzenie czy zmienna new tick jest prawdą. Ta zmienna jest prawdą, gdy aktualny tick jest tym, w którym operacje mogą być przeprowadzane. Jeśli mogą być to sprawdzane jest, czy jakiś przycisk jest aktualnie trzymany. Jest do tego wykorzystywana funkcja Is Hovered, która zwraca prawdę, gdy przycisk jest najeżdżany przez palec użytkownika. Jeśli któryś jest to zmienna Global Tick jest zmniejszany od 0,05. Zmienna Global Tick określa przedział czasowy między pojawieniem się New Tick. Jeśli jednak użytkownik nie trzyma żadnego przycisku to Global Tick jest resetowany do wartości 0,5.



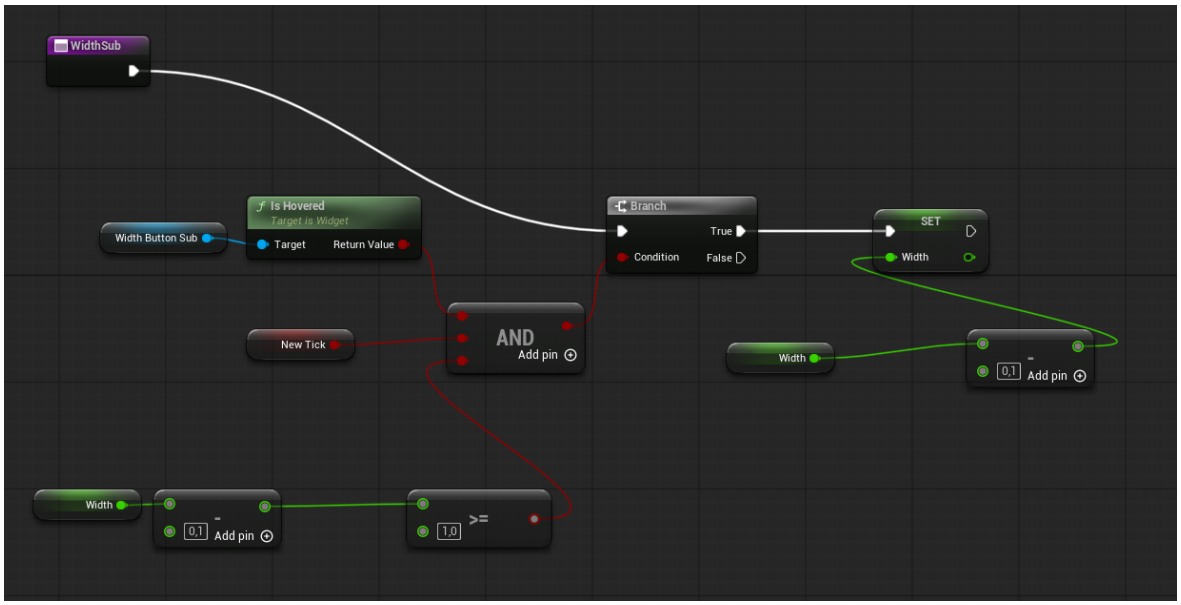
Rys. 3.26 Zmienne określające wymiary. Źródło: opracowanie własne

Te trzy zmienne odpowiadają kolejno za szerokość, długość oraz wysokość pomieszczenia.



Rys. 3.27 Funkcja Width Add. Źródło: opracowanie własne

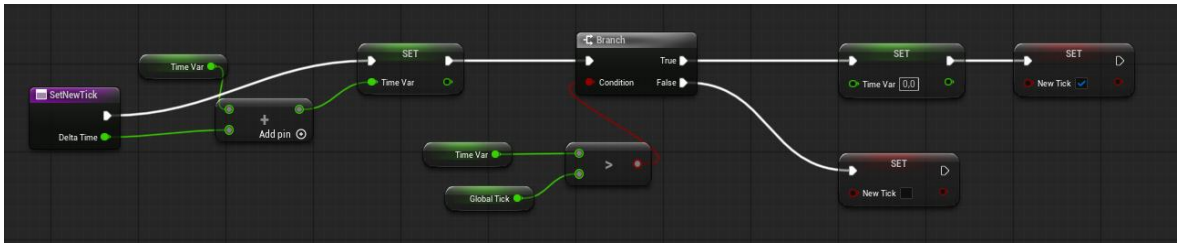
Funkcja Width Add zwiększa wartość zmiennej width, gdy odpowiedni przycisk jest trzymany. Blok Branch sprawdza czy odpowiedni przycisk jest trzymany, czy New Tick jest prawdą oraz czy wartość width po zwiększeniu nie będzie większa od 20. Jeśli wszystko będzie prawdą to zmienna width jest zwiększona o 0.1. Ustawienie limitu na 20 jest spowodowanie uniemożliwieniem użytkownikowi stworzenia zbyt dużego pomieszczenia.



Rys. 3.28 Funkcja Width Sub. Źródło: opracowanie własne

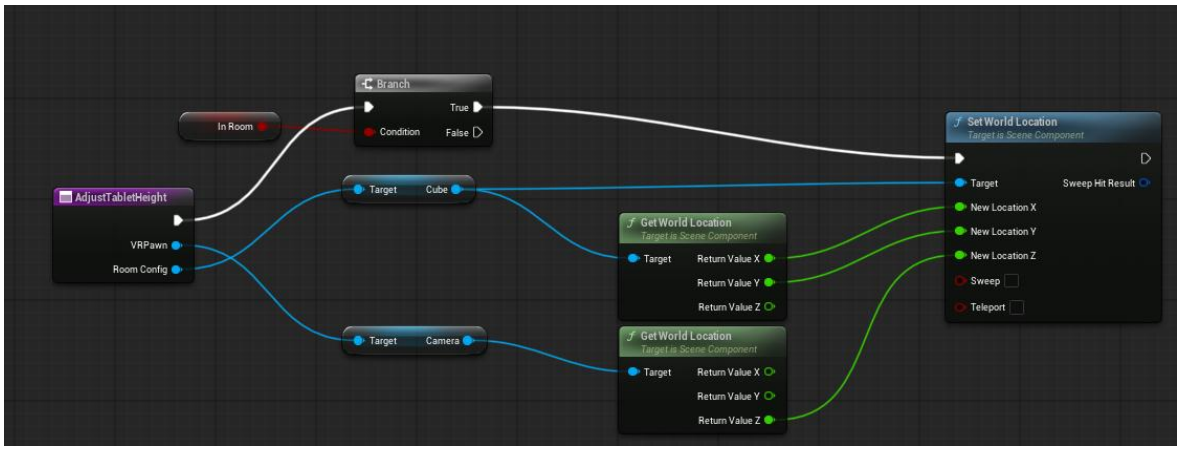
Na podobnej zasadzie działa proces zmniejszania wartości width. Tym razem minimalna wartość jaką może przyjąć funkcja wynosi 1. Z tego wynika, że do

modyfikacji jednej zmiennej potrzeba dwóch funkcji. Dlatego Event Tick przechodzi przez każdą z nich sprawdzając, czy któraś ze zmiennych ma być zmniejszona lub zwiększona.



Rys. 3.29 Funkcja Set New Tick. Źródło: opracowanie własne

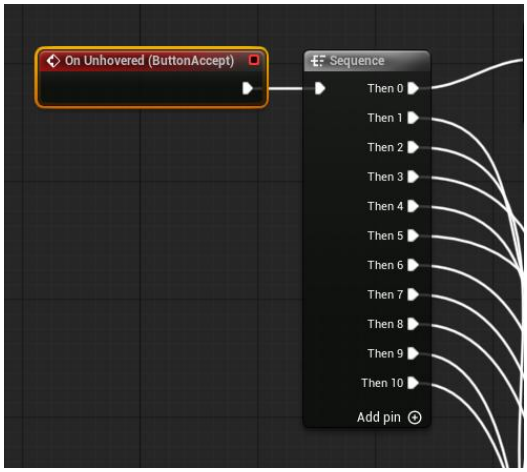
Funkcja Set New Tick jest odpowiedzialna za ustawienie zmiennej New Tick na true lub false. To ona determinuje, czy w aktualnym ticku mogą zostać wykonane działania. Cała funkcja opiera się o dwie zmienne. Time Var oraz Global Tick. Time Var kumuluje wartości przekazywane przez parametr Delta Time. Jest to zmienna odnosząca się do czasu pomiędzy dwoma tickami widgetu. Po dodaniu Delta Time do Time Var jest sprawdzany warunek, czy Time Var jest większy od Global Tick. Jeśli jest to Time Var jest resetowany i New Tick zostaje ustawiony na true. W przeciwnym wypadku New Tick jest ustawiany na false. Jako, że Global Tick jest zmniejszany, gdy któryś z przycisków jest trzymany to nadaje to efektu, że wartości zmiennych dodają się z coraz to większą prędkością.



Rys. 3.30 Funkcja Adjust Tablet Height. Źródło: opracowanie własne

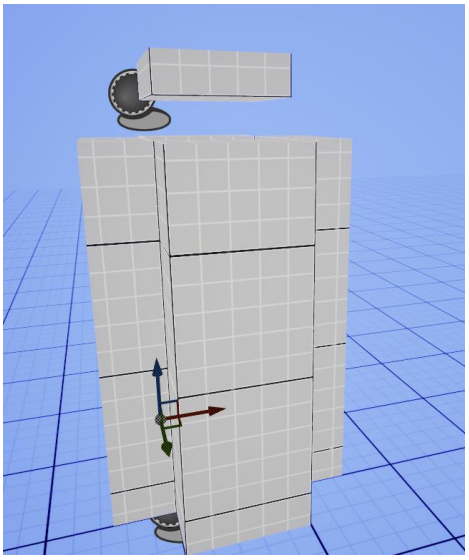
Zakładając, że każdy użytkownik ma inną wysokość to ekran z ustawieniami powinien się dostosowywać do poziomu oczu użytkownika. Funkcja Adjust Tablet Height jest odpowiedzialna za to zadanie. Za nim jednak zostaną przeprowadzone regulacje wysokości to jest sprawdzany warunek, czy zmienna In Room jest

prawdą. Ta zmienna jest ustawiana na true, gdy użytkownik jest w pomieszczeniu z ustawieniami, a na false, gdy nie jest. Gdy In room jest prawdą to wysokość Ekranu jest dostosowywana na podstawie pobranej wysokości kamery VRPawn.



Rys. 3.31 Blok typu event dla przycisku accept. Źródło: opracowanie własne

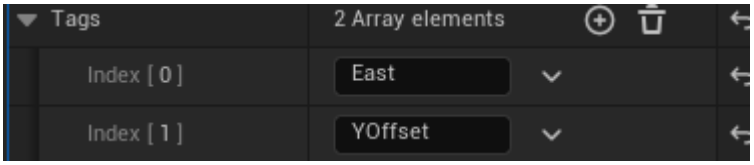
Ze względu na ilość operacji i bloków wyświetlanych na ekranie muszę podzielić omówienie tego segmentu na mniejsze części. Blok typu event On Unhovered aktywuje się, gdy użytkownik przestanie najeżdżać na przycisk Accept. Daje to iluzje interakcji dotykowej ekranu. Gdy blok zostanie wywołany to rozpocznie się proces generacji pomieszczenia. To co trzeba omówić to sposób w jaki pomieszczenie jest tworzone.



Rys. 3.32 Kształt pierwotny pomieszczenia. Źródło: opracowanie własne

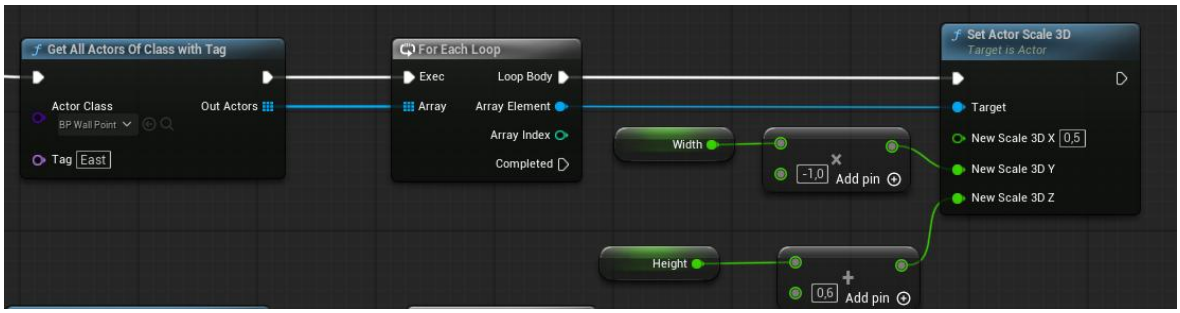
Pomieszczenie nie jest tworzone w czasie rzeczywistym ze względu na obciążenie wynikające z takiego procesu. Zamiast tego jest dostosowywany już umieszczony

na mapie składający się z 6 ścian pokój. Ściany są też specjalnie stworzonym blueprintem, ale ich implementacja będzie omówiona przy Stałych Obiektach. To co na ten moment jest kluczowe to ich tagi. Każda ze ścian ma specjalny tag, który określa w jaki sposób będzie modyfikowany ich rozmiar.



Rys. 3.32 Tagi dla wschodniej ściany.

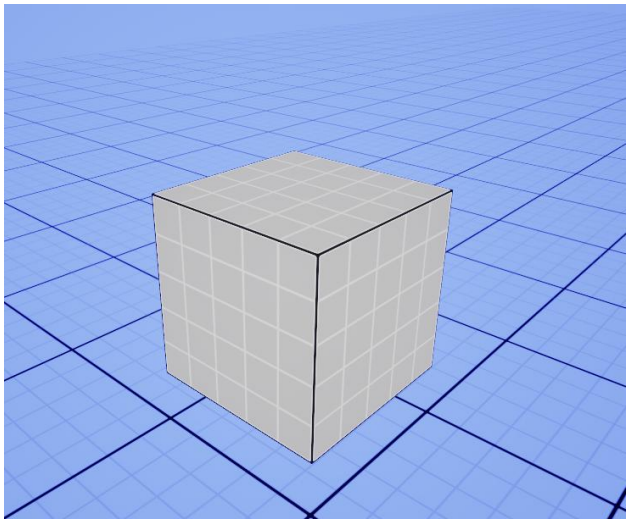
Są dwa rodzaje tagów. Jeden określa położenie ściany na mapie, a drugi określa czy ma być jego pozycja zmieniana. Na podstawie tych tagów te obiekty są odnajdywane oraz modyfikowane.



Rys. 3.33 Przykładowa modyfikacja skali wschodniej ściany. Źródło: opracowanie własne

Proces modyfikacji skali polega na odnalezieniu aktora ściany z tagiem East za pomocą Get All Actors Of Class with Tag. Jako że blok zwraca tablice wszystkich aktorów to trzeba po nich przejść przy pomocy bloku For Each Loop. Blok w każdej iteracji przechodzi po jednym aktorze z tablicy. Każdy aktor ma zmienianą skalę na podstawie zmiennych Width, Height oraz Length. Prostym założeniem jest określenie, że wartości x y z skali ściany odpowiadają kolejno Width, Length i Height. Jednakże tak nie jest z powodu potrzeby implementacji stałych obiektów, z której wynika, że ściana może mieć obiekt stały tylko na konkretnych stronach ściany. Dlatego każda z 4 głównych ścian jest obrócona na mapie, tak by dla użytkownika była widoczna tylko ta strona ściany, na którą można postawić ściane. Ściany z tagiem: East, North, South są kolejno obrócone o 90, 180 i -90 stopni. Przez co modyfikacja rozmiaru tych obiektów się różni dla każdej ściany. Jednak wywołanie bloku Set Actor Scale 3D jest identyczny dla każdego procesu.

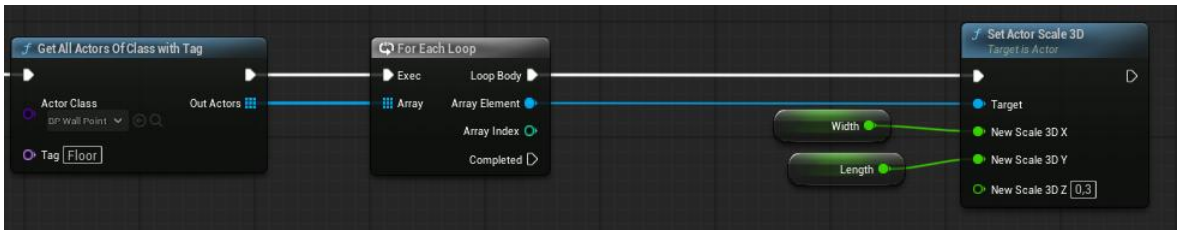
Modyfikowana zawsze jest oś Y i Z, a oś X jest stale ustawiona na 0,5. Jednym z założeń projektu jest odwzorowanie rzeczywistej skali w wirtualnym środowisku. Jako punkt odniesienia oraz podstawę do określenia skali użyłem obiektów wewnątrz Unreal Engine 5.



Rys. 3.34 Domyślna Kostka z domyślnym materiałem siatki. Źródło: opracowanie własne

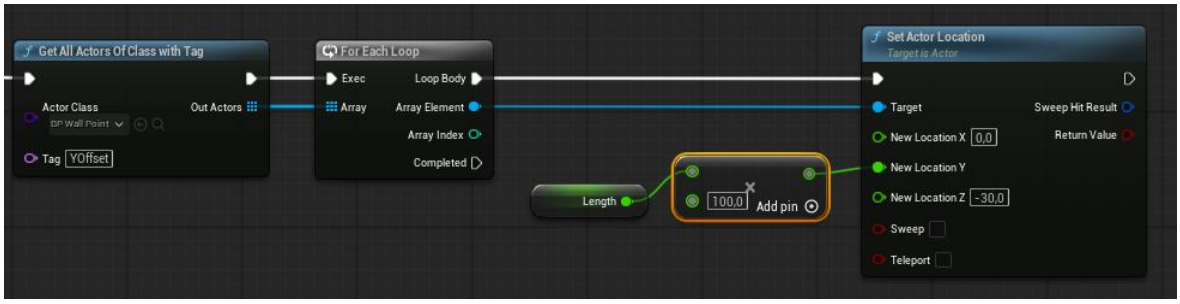
Unreal Engine 5 posiada domyślny materiał siatki, który dostosowuje się do położenia i rozmiarów obiektu. Dzieli on każdą stronę domyślnego sześcianu na 25 mniejsze kwadraty. Porównując swój rozmiar korzystając z VR oraz samą siatkę wyliczyłem, że każdy kwadrat siatki ma rozmiar 20 cm. W takim razie domyślna kostka ma rozmiary metr na metr na metr. Dodatkowo sprawdzając rozmiar przedmiotów domyślnych dostarczonych przez Unreal Engine 5 doszedłem do wniosku, że ustawienie skali przedmiotu powinno leżeć po stronie osoby dostarczającej model. Biorąc domyślną kostkę jako punkt odniesienia mogłem założyć, że skala jest podawana w metrach. Dla przykładu skala dla oś X ma wartość 1, więc obiekt ma po osi X rozmiar jednego metra. Centymetry są wartościami po przecinku skali. Skalę 1,23 można określić jako metr i 23 centymetry. Obliczanie położenia obiektu jest jeszcze prostsze. Badając wszystkie osie położenia obiektu wyszło, że jedna jednostka przesunięcia jest równa jednemu centymetrowi. W takim razie trzeba obiekt przesunąć od 100 jednostek, by odległość była równa jednemu metrowi. Zmienne Length, Width oraz Height są typu float oraz ich jednostka domyślna to metr. W takim razie podczas zmiany wartości 0.1 to wymiar zmienia się o jeden centymetr. Przy ustawianiu skali dla

ścian wystarczy przypisać odpowiednią zmienną do odpowiedniej ściany. Zmienna Width dotyczy ścian o tagu East oraz West. Związku z obrotem ścian na mapie samo przypisanie zmiennej do wartości skali daje nieoczekiwane rezultaty, jak dla przykładu rozmiar ściany się zmienia w złym kierunku. Dlatego dla tagu East zmienna width jest również mnożona przez wartość -1. W tym wypadku obie ściany rozszerzają się w odpowiednim kierunku. To samo tyczy się ścian North i South. Jest też tag określający pogłogę: Floor.



Rys. 3.35 Dostosowanie skali do podłogi i sufitu. Źródło: opracowanie własne

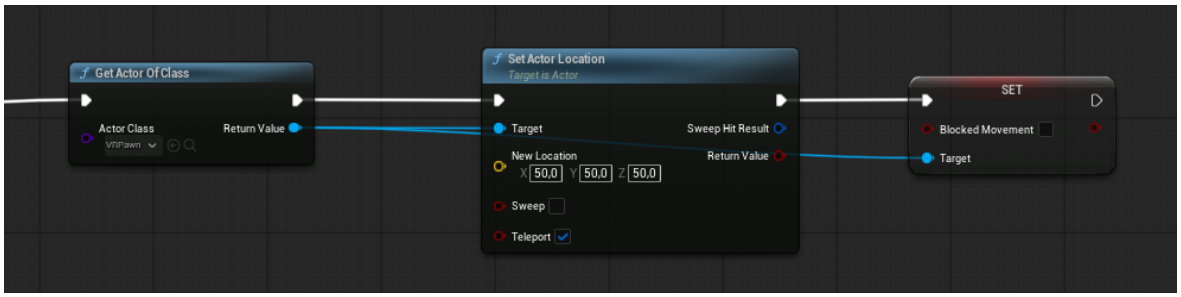
W tym wypadku wystarczy ustawić Width oraz Length dla kolejno osi X i Y. W ten sposób ściany z tagiem Floor rozszerzą się w obu osiach i dostosują się do rozmiarów ścian. Ten konkretny tag posiadają ściany reprezentujące podłogę oraz sufit. Kolejnym problemem podczas generacji pomieszczenia jest samo położenie ścian. One się rozszerzają, ale nie przemieszczają powodując, że pomimo zmian wartości to dla użytkownika pokój ma ten sam rozmiar.



Rys. 3.36 Dostosowanie położenia dla wschodniej ściany. Źródło: opracowanie własne

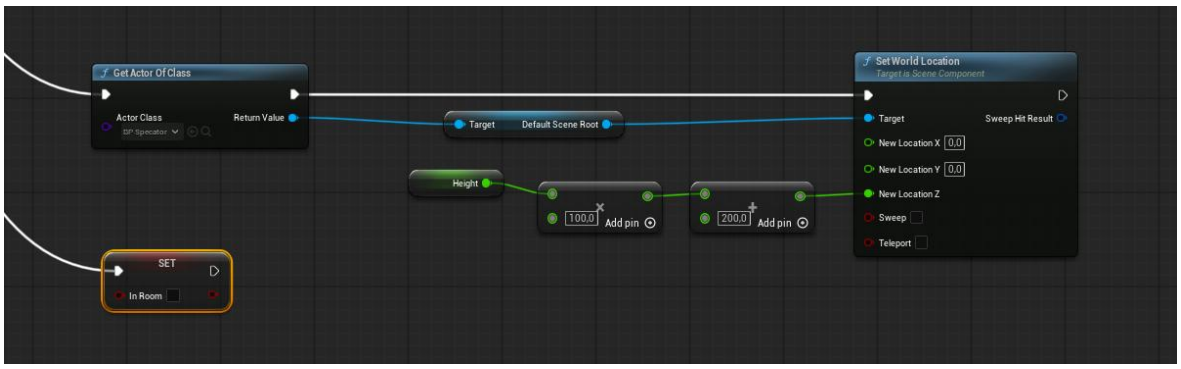
Sufit, północna ściana oraz zachodnia ściana mają przypisana dodatkowo kolejno tagi: ZOffest, XOffset, YOffset. Te ściany muszą zmienić położenie po rozszerzeniu ścian. Tu operacja jest bardzo prosta. Po znalezieniu aktorów z konkretnym tagiem są im przypisywane nowe lokalizacje na podstawie zmiennych Width, Length, Height. Są one dodatkowo mnożone przez 100 ze względu na wcześniej omawianą skalę jednostek. Dla tagu YOffset jest używana zmienna

Length, dla XOffset zmienna Width, a dla ZOffset zmienna Height. W ten sposób program generuje całe pomieszczenie na podstawie wytycznych użytkownika.



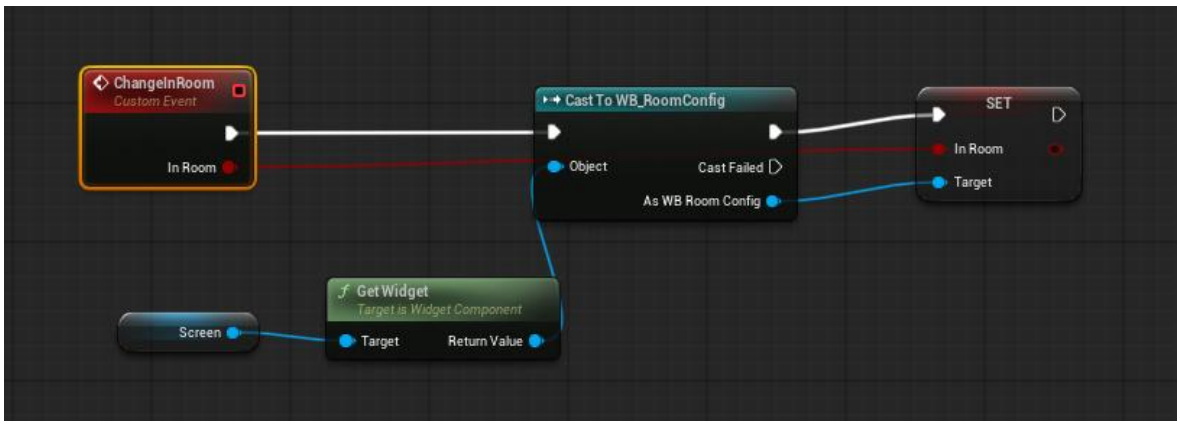
Rys. 3.37 Przeniesienie użytkownika do pomieszczenia. Źródło: opracowanie własne

Po generacji pomieszczenia lokalizacja użytkownika jest przemieszczana na koordynaty 50 50 50. Jest to stały punkt, gdzie użytkownik zawsze będzie przenoszony po generacji pomieszczenia. Na koniec też jest zmieniana wartość Blocked Movement w aktorze VRPawn odblokowujący możliwość poruszania się po przestrzeni 3D.



Rys. 3.38 Ustawienie obserwatora oraz wartości In Room. Źródło: opracowanie własne

Nie będę w tym segmencie omawiał funkcjonalności obserwatora z lotu ptaka, ale ważnym elementem jest ustawienie jego pozycji startowej. Jego wysokość jest ustawiana na wysokość całego pomieszczenia z dodatkowym oddaleniem dwóch metrów. Na sam koniec procesu generacji zmienna In Room jest ustawiana na wartość false oznaczającą, że użytkownik nie znajduje się aktualnie w pokoju ustawień.



Rys. 3.39 Zmiana zmiennej In Room. Źródło: opracowanie własne

Ostatnią ważną częścią kodu jest wystawienie punktu końcowego dla zmiany wartości In Room. Ten punkt jest wystawiony dla obiektu BP_RoomConfigTablet, który zawiera widget z tą zmienną. Dlatego, by zmienić ją trzeba się odwołać wyłącznie do tego obiektu. Po wywołaniu pobierany jest widget z ekranu i jest przetwarzana operacja rzutowania pobranego widgetu na konkretną klasę. Po udanej operacji zmienna zostaje zmieniona na przekazaną jako argument.

3.3 Tablet

3.4 Tworzenie Przedmiotów

3.5 Zmiana Wyglądu Ścian

3.6 Stałe Obiekty

3.7 Ustawienia